Spatiotemporal forecast with deep learning

Course: Programming applied to cartographic engineering

Maurício C. M. de Paulo - D.Sc.

November 11, 2026

Outline



1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

Outline

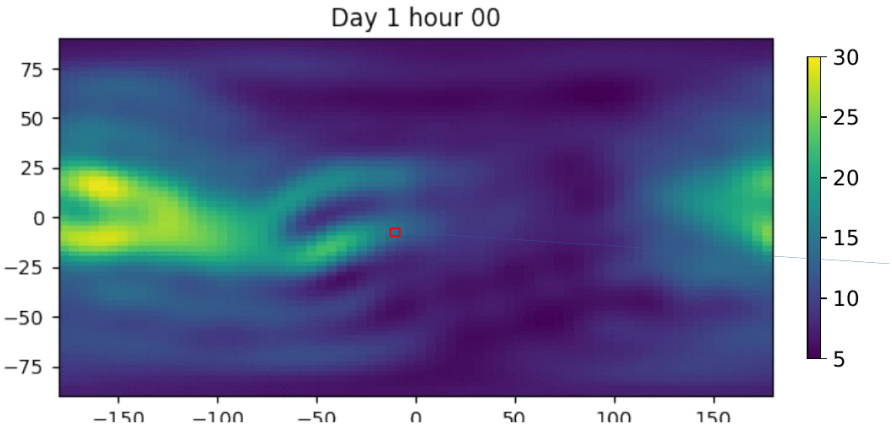


1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

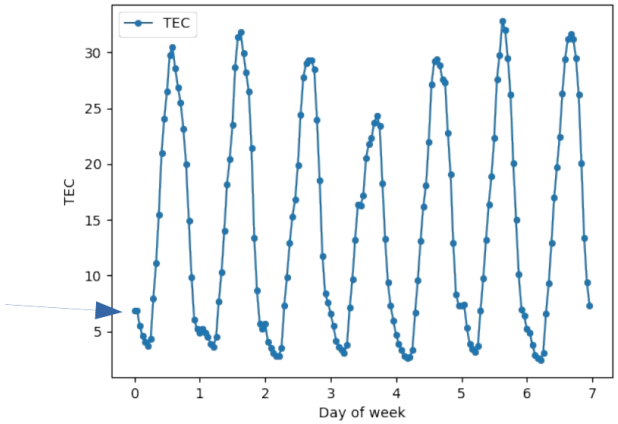
Global ionosphere maps prediction

Objective: Share my experience with deep learning for ionosphere maps forecast.

Predict the next maps in a time series.



Ionosphere maps (VTEC) of a single day (single band, but colored).



VTEC time series of a single pixel of the map along an entire week.

`https://github.com/mauriciodev/tec_forecast` (Tensorflow 2/Keras)

Paulo MCM de, Marques HA, Feitosa RQ, Ferreira MP (2023) New encoder–decoder convolutional LSTM neural network architectures for next-day global ionosphere maps forecast. GPS Solut 27(2):95.

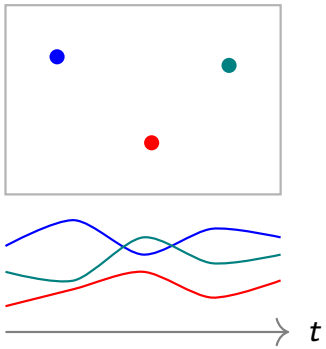
`https://doi.org/10.1007/s10291-023-01442-4`

Outline

1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

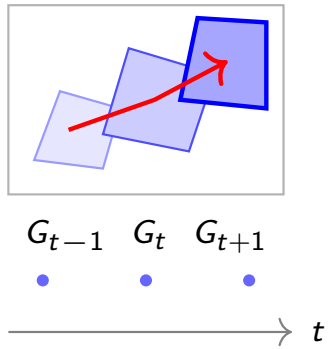
Types of spatiotemporal problems

1. Coordinates in time



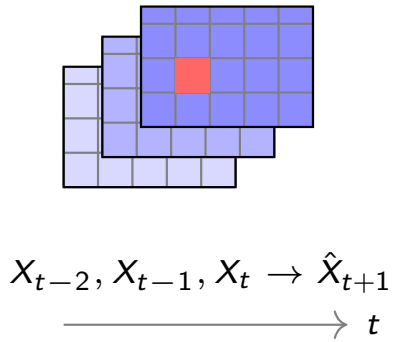
- Fixed stations / sensors (GNSS, weather, traffic).
- Tensor: (T, N, F)
  N locations, F variables
- **Multivariate time series** (LSTM, TCN, transformers).
- Neighborhood as a graph: ST-GNN.

2. Geometries in time



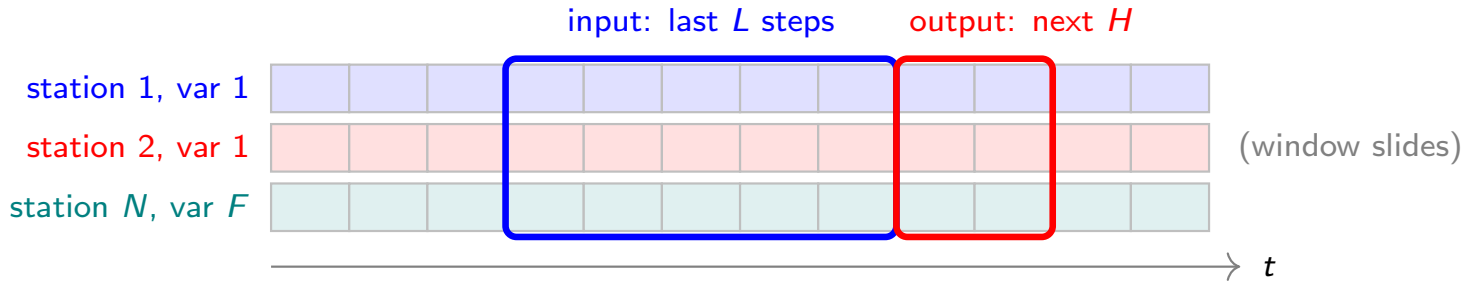
- Moving / deforming objects: trajectories, storms, fire fronts, glaciers.
- Variable size: sequences of vertices or polygons.
- Not addressed in this course.

3. Maps / images in time



- Regular grids: VTEC maps, radar, satellite, reanalysis.
- Tensor: (T, C, H, W)
- **Video prediction** (ConvLSTM, 3D CNN, SimVP, transformers).
- Focus of this course.

1. Coordinates in time: multivariate time series



Data

- Stations at fixed coordinates: GNSS (VTEC), weather, air quality, traffic.
- Table $(T, N \times F)$: each location/variable is a series.

Strategies

- *Local*: one model per series.
- *Global*: one model for all series (coordinates as covariates).
- *Multivariate*: all series jointly (cross-correlations).

Models

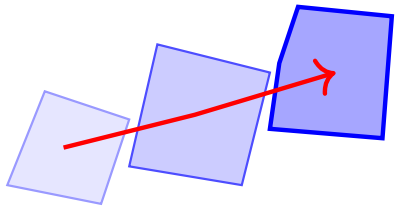
- LSTM/GRU, TCN, N-BEATS/N-HiTS, TFT, PatchTST.
- Library: `darts` (same API for all).

Using space explicitly

- Neighbors as a graph $\rightarrow$ spatiotemporal GNNs (DCRNN, Graph WaveNet).
- Dense stations $\rightarrow$ interpolate to a grid (approach 3).

(Herzen et al. 2022; Nie et al. 2023; Li et al. 2018; Wu et al. 2019)

2. Geometries in time



G_{t-2}, G_{t-1}, G_t

vertices $[(x_1, y_1), \ldots, (x_k, y_k)]$

trajectory of the centroid

rasterized mask / distance field

Examples

- Moving points: hurricane tracks, vessels (AIS), vehicles, animals.
- Deforming shapes: storm cells, fire fronts, flood extent, glacier fronts.

Why it is hard

- Variable number of vertices per object.
- Objects split, merge, appear and vanish.
- Needs tracking (object identity over time).

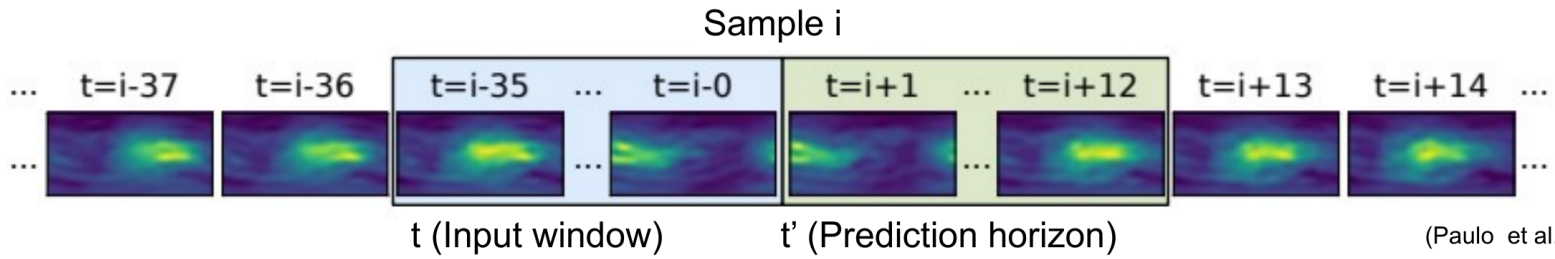
Common workarounds

- Trajectories → sequence models on coordinates (Social LSTM, transformers).
- Represent bounding boxes with x, y, height, width as features (approach 1).
- Shapes → rasterize, predict maps (approach 3), vectorize again.

Whole geometry forecast will not be addressed in this course.

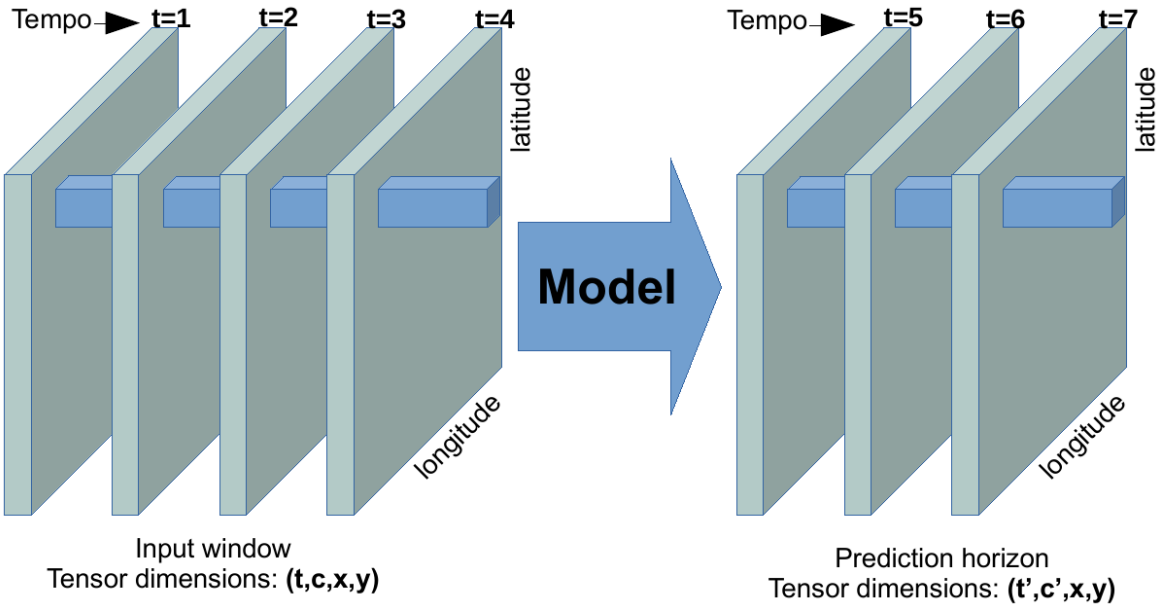
(Alahi et al. 2016)

3. Maps in time: predicting the next maps



(Paulo et al. 2023)

Sliding window sampling procedure.



Main loss functions:

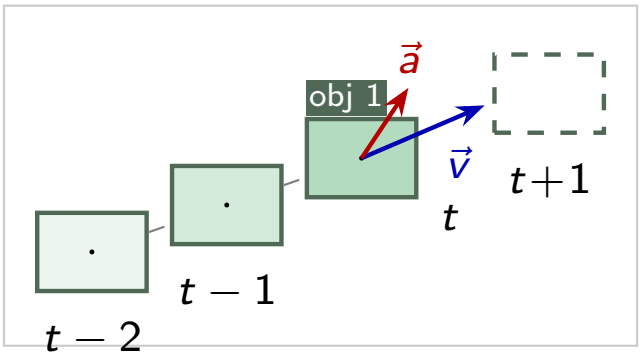
$$MAE = \frac{\sum_{i=1}^{n} |y_i - \hat{y}_i|}{n}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}}$$

The model maps t input images (t, C, H, W) to the next t' images (t', C, H, W).

Tracking objects vs. forecasting the whole field

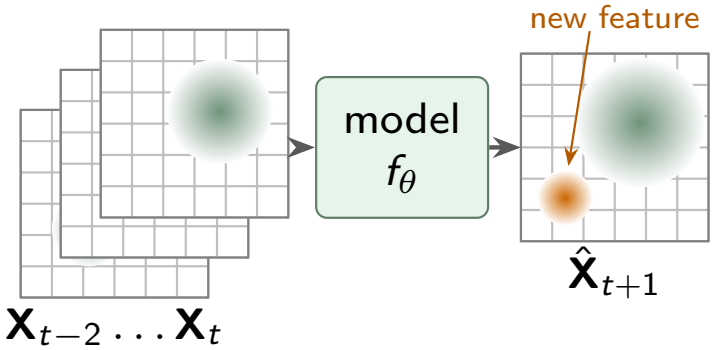
Object kinematics



$$\hat{\mathbf{x}}_{t+1} = \mathbf{x}_t + \vec{v}\,\Delta t + \tfrac{1}{2}\vec{a}\,\Delta t^2$$

- Detect boxes → track → fit $\vec{v}, \vec{a}$ → extrapolate.
- Objects keep their shape and intensity; few, interpretable parameters.
- Fails when objects grow, decay, split, merge or appear.

Spatiotemporal forecast



$$\hat{\mathbf{X}}_{t+1} = f_\theta(\mathbf{X}_{t-k}, \ldots, \mathbf{X}_t)$$

- Predicts every pixel: no detection or tracking.
- Learns motion *and* change: growth, decay, initiation.
- Needs training data; MSE/MAE training blurs the forecast.

Bridge: optical flow

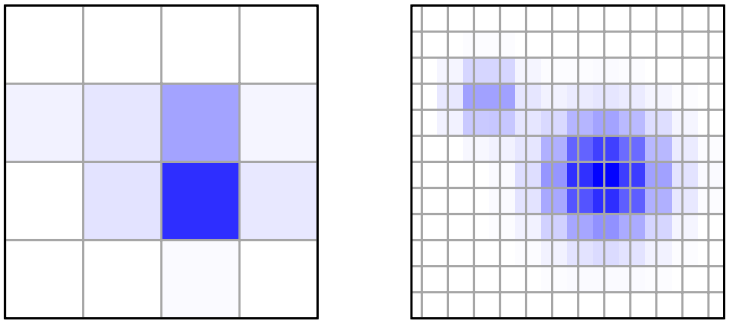
A velocity for *every pixel* (kinematics of the whole field). ConvLSTM and SimVP learn this motion **plus** how the values change along it.

Outline

1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

Building the time series: resolution

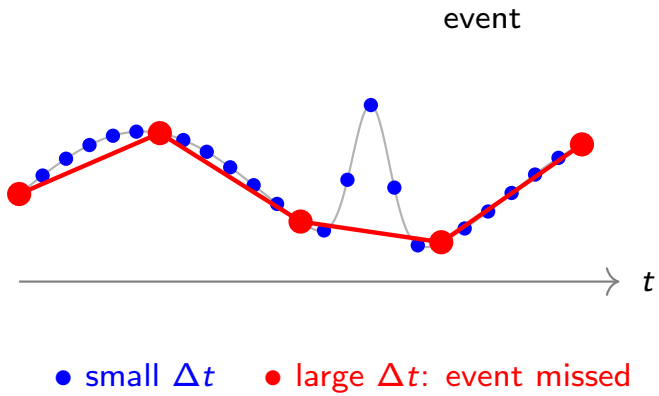
Spatial resolution (H, W)



coarse (4×4) fine (12×12)

- Coarse grids lose small structures (the small blob).
- Memory grows with $H \times W$ ($4\times$ for half the cell size).
- With n pooling layers, H and W must be multiples of 2^n: crop or pad.
  e.g. $72 \to 36 \to 18 \to 9$ ($n = 3$)

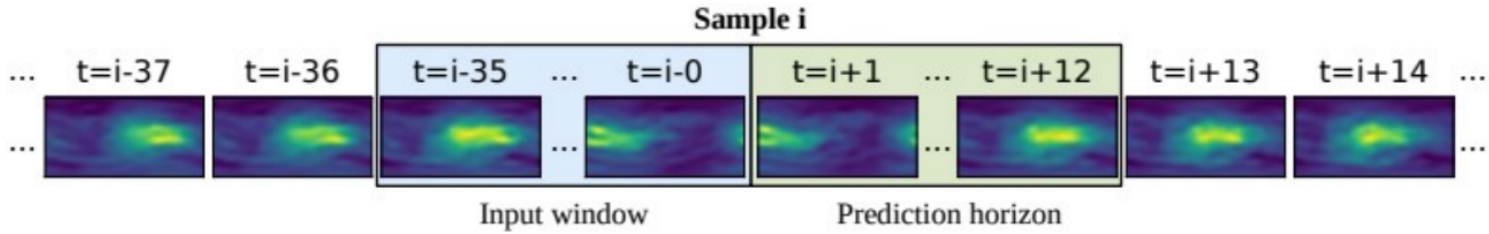
Temporal resolution (Δt)



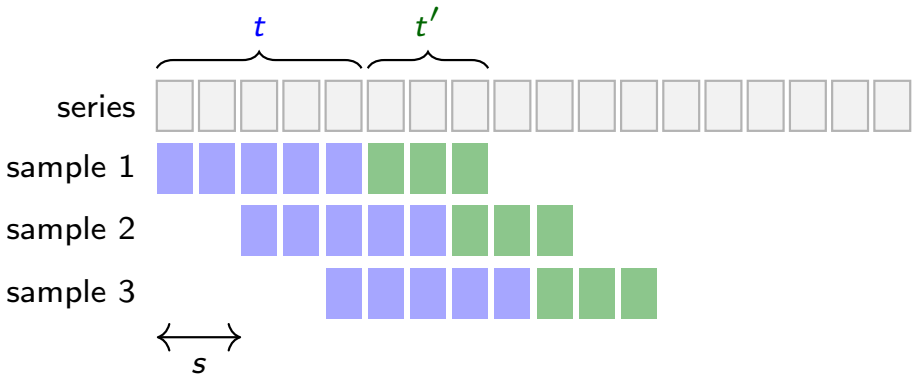
- Δt must be compatible with the event: at least 2 samples per shortest period of interest (Nyquist).
- Smaller $\Delta t \to$ more steps for the same horizon (longer sequences).
- Resample: average (smooth) or instantaneous values.

Result: one tensor (T, C, H, W) for the whole period, ready to be sliced into samples.

Sliding window: input length, horizon and stride



One sample from the VTEC series (Paulo et al. 2023).



Number of samples: $N = \left\lfloor \frac{T-t-t'}{s} \right\rfloor + 1$

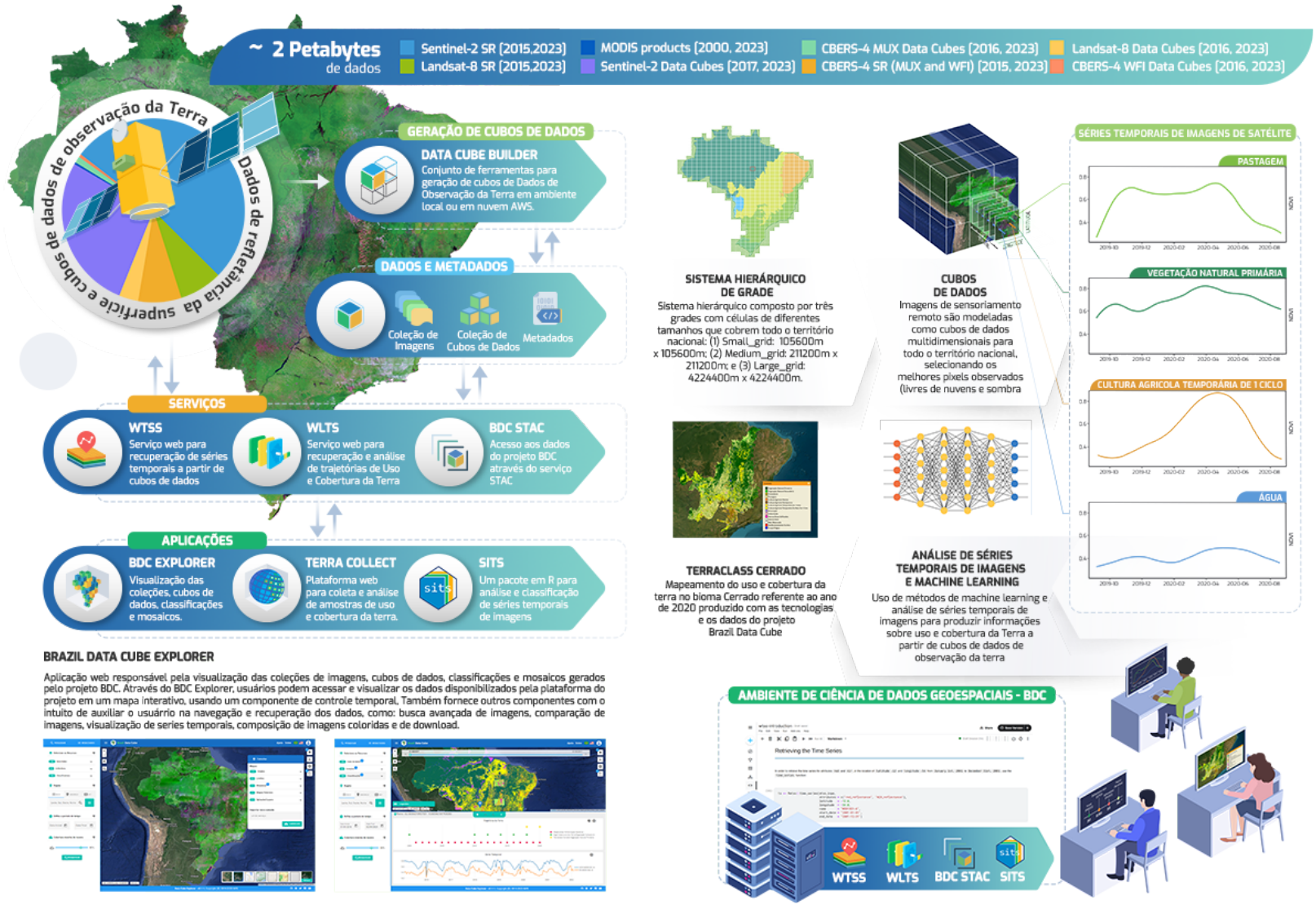
- **Input length** t: long enough to describe the dynamics (e.g. a full daily cycle).
- **Output length** t': long enough to be usable.
- **Stride** s: small $s \rightarrow$ more samples, but highly correlated.
- Tensors: input (t, C, H, W), output (t', C', H, W).
- Split train/val/test by time blocks: no window crosses two sets.

Obtaining remote sensing image time series

	Google Earth Engine	**STAC** (e.g. Microsoft Planetary Computer)	**Brazil Data Cube** (INPE)
What it is	Catalog + processing platform	Open standard to describe and search imagery; files in the cloud (COG, Zarr)	STAC catalog + analysis-ready data cubes of Brazil
Where you compute	Google servers: you build a lazy graph (`ee.Image`)	Your machine or cluster: reads only the needed chunks (Dask)	Your machine; time series extracted on the server (WTSS)
Python	`earthengine-api`, `geemap`, `xee` ($\rightarrow$ xarray)	`pystac-client`, `planetary-computer`, `odc-stac` / `stackstac`	`pystac-client`, `wtss`, `wlts`, `odc-stac`
Access	Cloud project; free for research, quotas and export limits	Free and open; URLs signed by `planetary_computer`	Free; access token
Strength	Huge catalog, planetary scale, no download	Same code for any STAC (AWS Earth Search, Copernicus, BDC...)	Regular cubes (16 days, monthly), CBERS, ready for time series
Limitation	Vendor lock-in; exporting tensors for DL is slow	You handle compute, cloud masks and compositing	Brazil only; fewer sensors

Common workflow: search (collection, bbox, dates, clouds) $\rightarrow$ lazy load $\rightarrow$ cloud mask $\rightarrow$ composite / resample to a regular Δt $\rightarrow$ tensor (T, C, H, W)

Brazil Data Cube



Main image time series (data cubes)

System	Spatial	Temporal	Bands represent	Main applications
Sentinel-1 (C-band SAR)	10 m	6–12 days	VV, VH backscatter: roughness, structure, moisture. Sees through clouds, day and night	Floods, deforestation in cloudy regions, crops, soil moisture, InSAR deformation
Sentinel-2 (MSI)	10 / 20 / 60 m	5 days	13 bands: visible, red-edge, NIR, SWIR (+ aerosol, water vapor, cirrus)	Vegetation indices, land cover, agriculture, water quality
Landsat 8/9 (OLI/TIRS)	30 m (15 m pan, 100 m thermal)	16 days each, 8 combined	Visible, NIR, SWIR + 2 thermal (surface temperature)	Long-term change (archive since 1984), deforestation history, urban heat
MODIS (Terra/Aqua)	250 / 500 / 1000 m	1–2 days; 8 / 16-day composites	36 bands, visible to thermal IR	NDVI/EVI (MOD13Q1), LST, fire, snow, phenology (successor: VIIRS)
GOES-East (ABI, GOES-16/19)	0.5–2 km	10 min (full disk), 1 min (mesoscale)	16 bands: 2 visible, 4 NIR, 10 IR (water vapor, cloud-top temperature)	Nowcasting, convection, rainfall estimation, fire, cloud motion
CBERS-4/4A (WFI)	64 m	5 days	Blue, green, red, NIR	Brazilian monitoring; in BDC cubes
WorldView-3 (Maxar, commercial)	0.31 m pan, 1.24 m MS, 3.7 m SWIR	$<$1 day, on demand (irregular series)	Pan + 8 VNIR (coastal, yellow, red-edge) + 8 SWIR	Urban mapping, infrastructure, object detection, minerals

- Trade-off: fine pixels $\leftrightarrow$ frequent revisit. Prefer Level-2 (surface reflectance) with a cloud mask.
- All except WorldView-3 (commercial) are in GEE and Planetary Computer. BDC: Sentinel-2, Landsat, CBERS and MODIS cubes over Brazil.

Brazil Data Cube in Python

Library	Provides
`pystac-client` + `odc-stac`	Search the BDC STAC and load a **datacube** (xarray: time, band, y, x)
`wtss`	**1D time series** of a cube at a point, extracted on the server (no image download)
`wlts`	Land use/cover **trajectory** at a point (PRODES, TerraClass, MapBiomas): categorical 1D series, useful as labels
`wcpms`	Phenology metrics (start, peak, end of season) of a point
`sits` (R)	Full workflow: cubes, samples, classification

Cube names: `S2-16D-2`, `LANDSAT-16D-1`, `CBERS4-WFI-16D-2`... A free access token is required.

1D time series (WTSS)

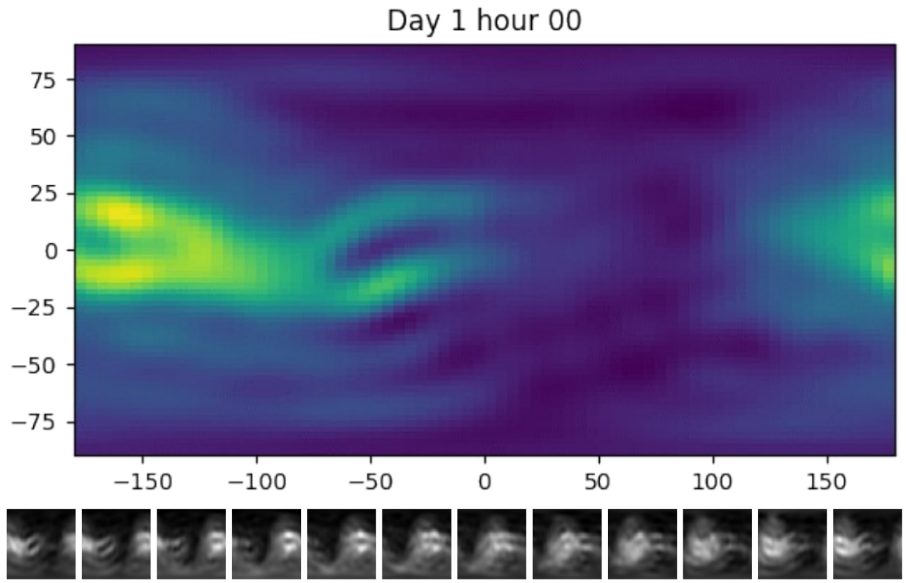
```
from wtss import WTSS
service = WTSS('https://data.inpe.br/bdc/wtss/v4/',
               access_token=TOKEN)
cov = service['S2-16D-2']
ts = cov.ts(attributes=('NDVI', 'EVI'),
            latitude=-12.0, longitude=-54.0,
            start_date='2020-01-01',
            end_date='2023-12-31')
ts.plot()
```

Datacube (STAC)

```
import pystac_client, odc.stac
url = 'https://data.inpe.br/bdc/stac/v1/'
cat = pystac_client.Client.open(url)
items = cat.search(collections=['S2-16D-2'],
                   bbox=[-54.1, -12.1, -53.9, -11.9],
                   datetime='2022-01-01/2022-12-31')
ds = odc.stac.load(items.item_collection(),
                   bands=['B04', 'B08', 'NDVI'], chunks={})
cube = ds.to_array('band').transpose('time', 'band',
                                     'y', 'x')  # (T, C, H, W)
```

Exploratory analysis: what to look for

	Look for	Decides
Values	range, skewness, outliers, gaps	normalization, loss
Time	trend, seasonality, noise, events	Δt, t, t', split
Space	gradients, clusters, borders	H, W, crop / pad
Space-time	propagation, lags	architecture, t

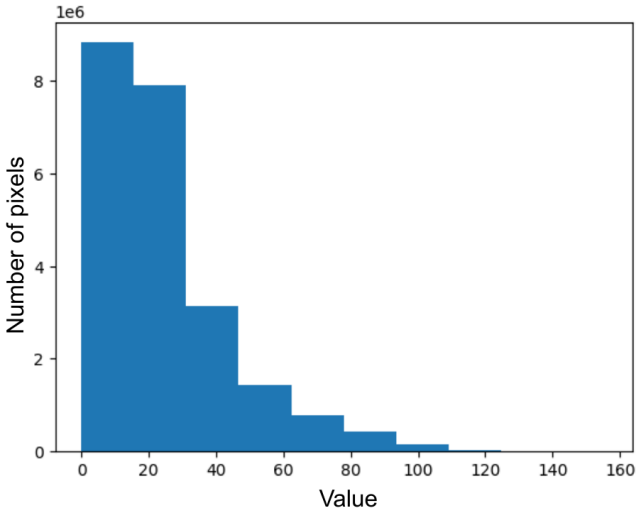


Watch the sequence first: the VTEC peak follows the Sun westward.

Defining hyperparameters

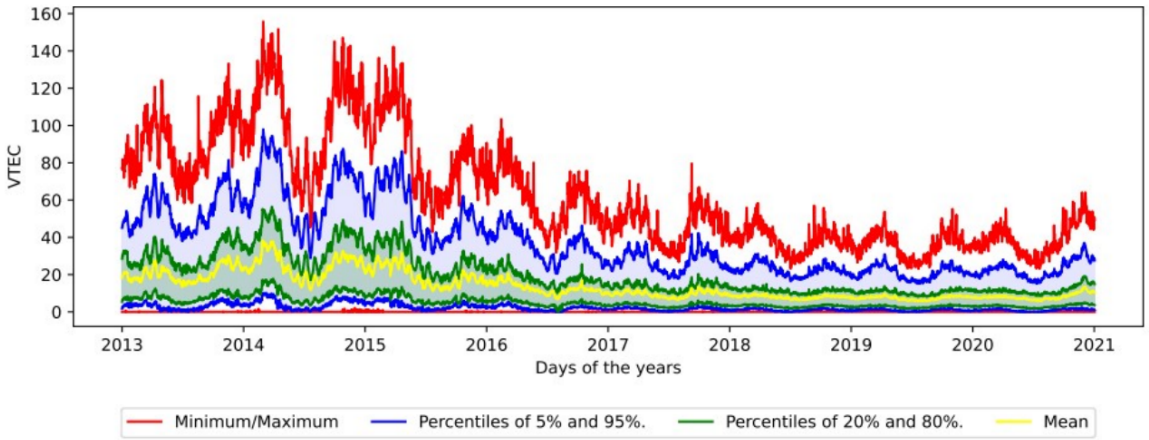
Exploratory analysis + previous research + a simple search (it does not have to be fancy).

Exploratory analysis: distributions



Histogram of all pixels:
$(T, C, H, W) \rightarrow (T \cdot H \cdot W, C)$

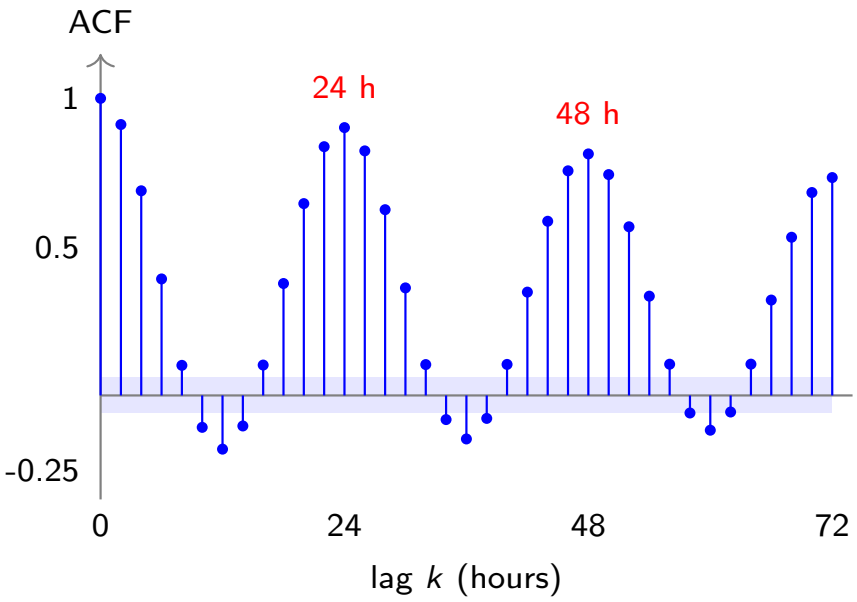
- Right-skewed, non-negative.
- Min-max or log normalization; robust losses.



Percentiles of each map over time: $(T, C, H, W) \rightarrow (T, C \cdot H \cdot W)$

- Trend: solar cycle 24 (maximum in 2014, minimum in 2019–2020).
- Seasonality: annual and semiannual oscillations.
- Non-stationary: a test set from 2020 has much lower values than training data from 2014 $\rightarrow$ split and normalize with care.

Temporal correlation: autocorrelation function (ACF)



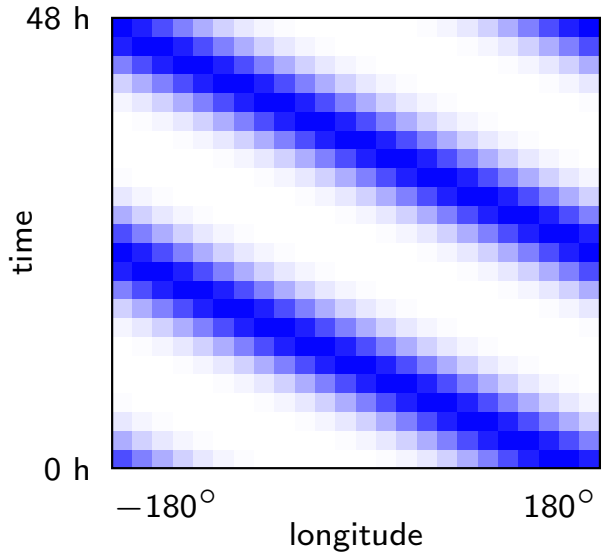
Illustrative ACF of an hourly VTEC pixel.
Shaded: 95% band of white noise.

$$\rho(k) = \text{corr}(x_t,\, x_{t+k})$$

- **Peaks** $\rightarrow$ seasonal period (24 h): t should cover at least one cycle.
- **Decay** $\rightarrow$ how far ahead the past is informative (t').
- **PACF**: direct effect of lag k, removing intermediate lags.
- **For maps**: ACF of each pixel, or $\text{corr}(X_t, X_{t+k})$ over all pixels = skill of the *persistence* baseline $\hat{X}_{t+k} = X_t$.

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(cube[:, 0, i, j], lags=72)   # cube: (T, C, H, W), pixel (i, j)
```

Spatiotemporal tools



Illustrative Hovmöller diagram at the equator: the slope shows the westward motion of the VTEC peak (15°/h).

Tool	Reveals	Python
Hovmöller diagram	propagation direction and speed	`xarray` + `matplotlib`
Mean / std maps	where the signal and the variability are	`xarray`
Correlation map (vs. a reference pixel, with lag)	teleconnections, delays between regions	`xr.corr`
Moran's I / variogram	spatial autocorrelation and its range	`esda` (PySAL), `scikit-gstat`
EOF / PCA	dominant spatial modes + their time series	`xeofs`, `scikit-learn`
STL decomposition (per pixel or region)	trend, seasonality and residuals	`statsmodels`

Outline

1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

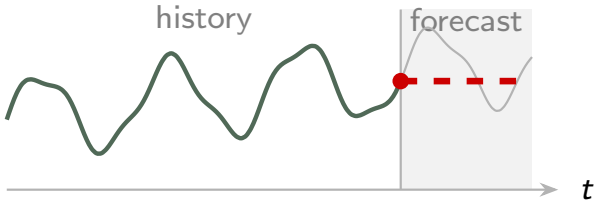
Architectures



- Baselines (what every model must beat)
- Recurrent neural networks
- 3D Convolutional networks
- 2D Convolutional networks (based on SimVP)
- Transformers networks
- Generative models: diffusion and flow matching

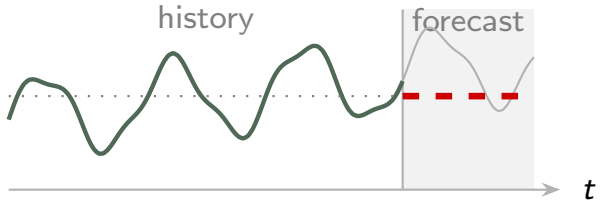
Baselines: what every model must beat

Persistence (stationary)



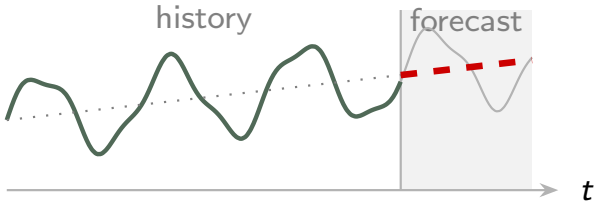
$\hat{y}_{t+h} = y_t$: tomorrow equals now

Climatology (mean)



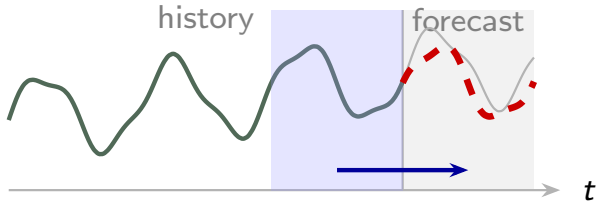
$\hat{y}_{t+h} = \bar{y}$: average of the past

Linear regression (trend)



$\hat{y}_{t+h} = a + b\,(t+h)$: extend the trend

Repeat the last cycle (seasonal naive)



$\hat{y}_{t+h} = y_{t+h-P}$: $P = 1$ day or 1 year

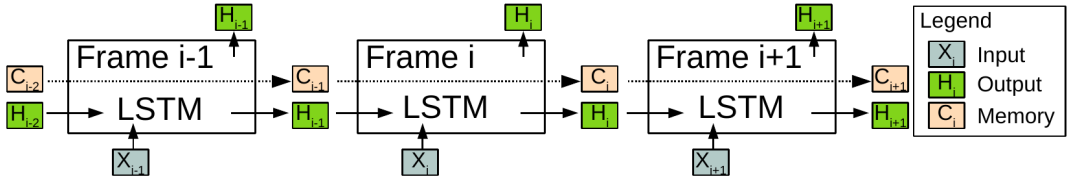
Use them as the reference

TEC: repeat **yesterday** ($P = 1$ day). Vegetation, land cover: repeat **last year** ($P = 1$ year).
Skill score: $SS = 1 - \mathrm{MSE}_{model}/\mathrm{MSE}_{baseline}$ ($SS > 0$: the model beats the baseline).

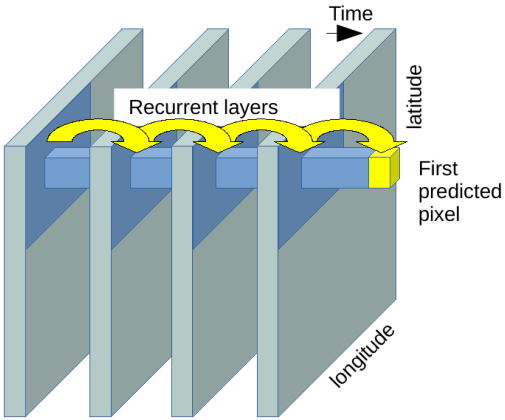
Recurrent neural networks: ConvLSTM

$H_i = f(X_i, H_{i-1}, C_{i-1})$
With: $H_0 = 0$ e $C_0 = 0$

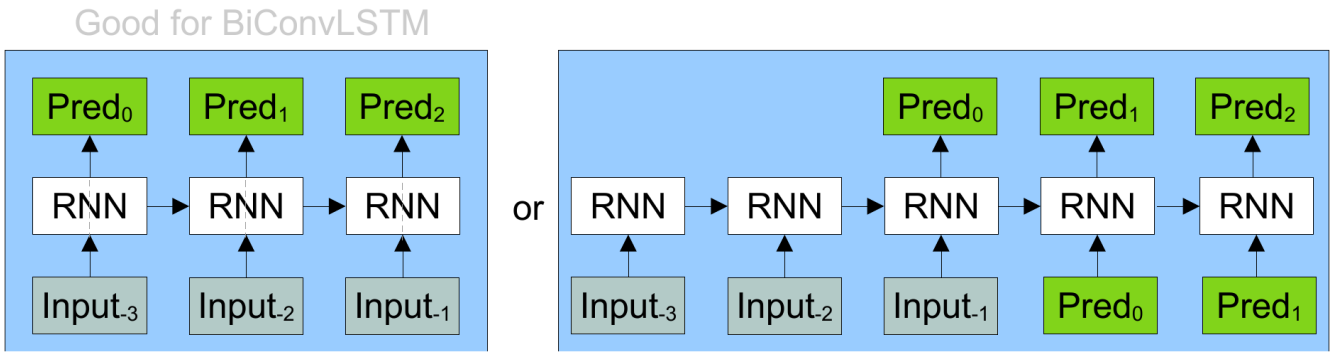
X_i: input data of current frame;
H_i: output of current frame;
H_{i-1}: output of the previous frame;
C_{i-1}: memory cell of the previous frame.



Recurrent LSTM cell processing for frames X_{i-1}, X_i e X_{i+1}



Single step prediction with RNN



Prediction methods with one RNN

Recurrent neural networks: ED-ConvLSTM

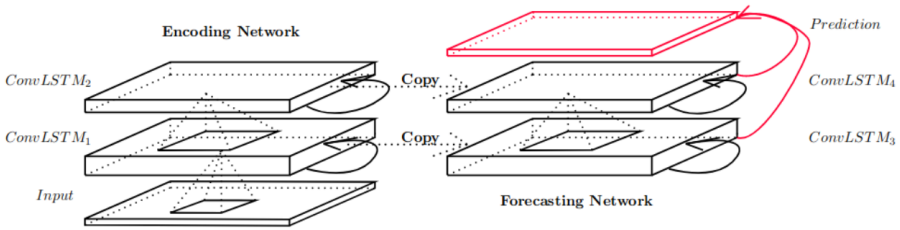
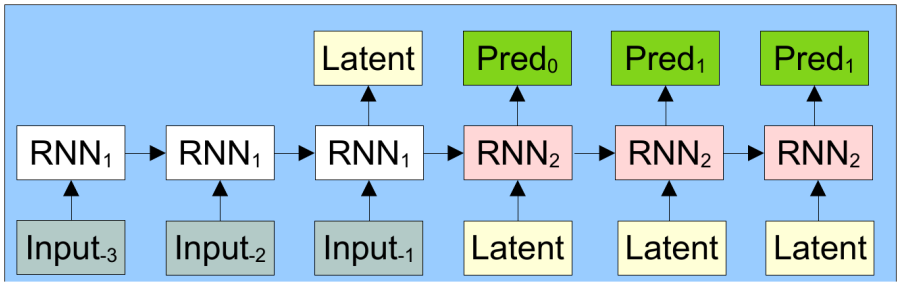


Figure 3: Encoding-forecasting ConvLSTM network for precipitation nowcasting

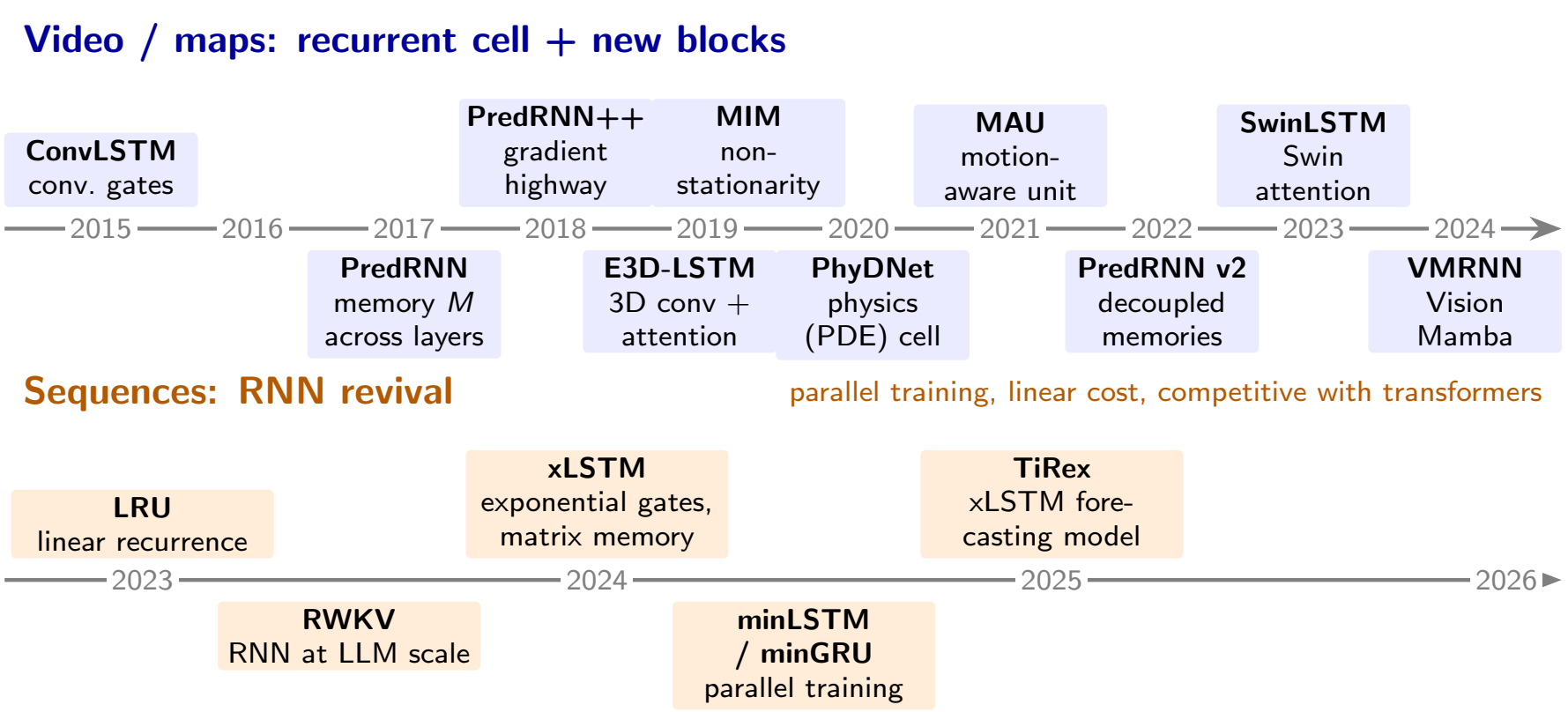
ConvLSTM was actually first designed with different layers for encoding and forecasting. (Shi et al. 2015)



Prediction method with Encoder-Decoder ConvLSTM.

Most articles ignore the difference between ConvLSTM and ED-ConvLSTM, but there are articles that show the error propagation.

Recurrent Neural Networks evolution

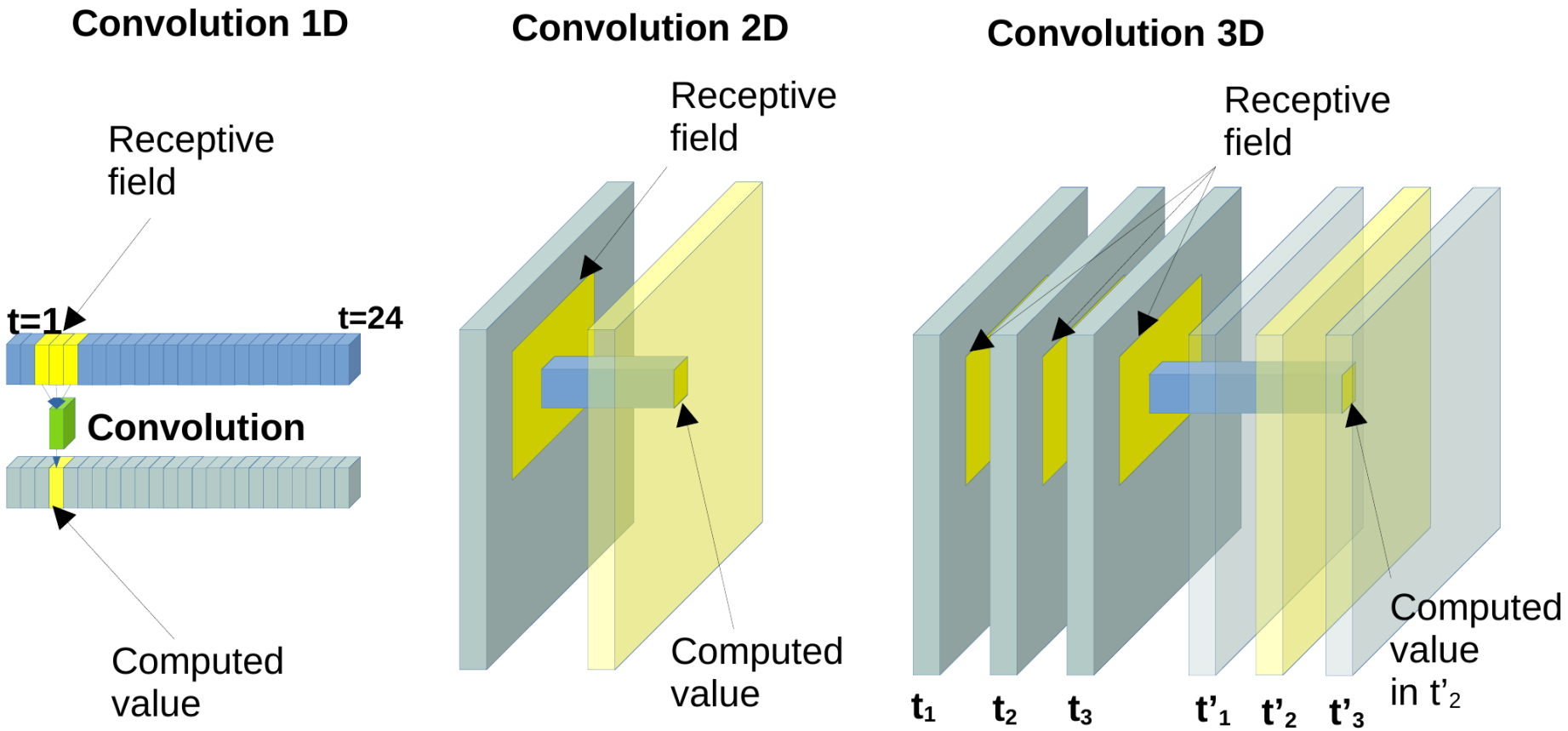


Where are we?

Video RNNs now borrow blocks from transformers and SSMs (SwinLSTM, VMRNN). For sequences, xLSTM is strong (TiRex leads zero-shot forecasting benchmarks), but a **convolutional xLSTM for video prediction** has not been published yet.

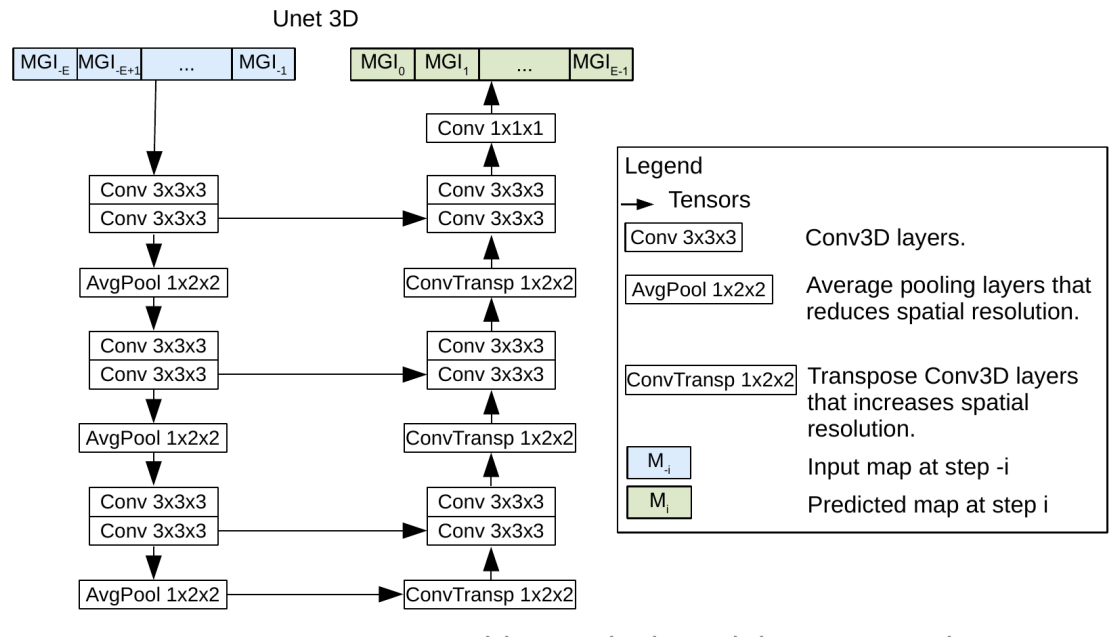
Video models: OpenSTL GitHub page; Tang et al. 2024. Sequences: Orvieto et al. 2023; Peng et al. 2023; Beck et al. 2024; Feng et al. 2024; Auer et al. 2025.

Remembering convolutional layers



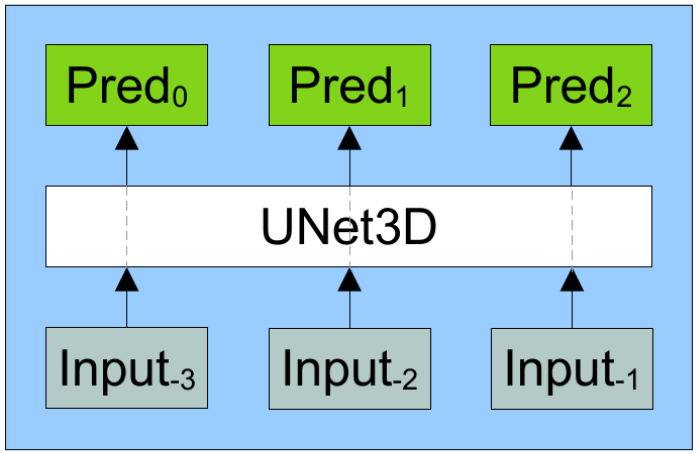
Obs.: 3D convolutions with stride 1 and pad do not modify the t, x, y dimensions.

3D Convolutional networks: UNet 3D



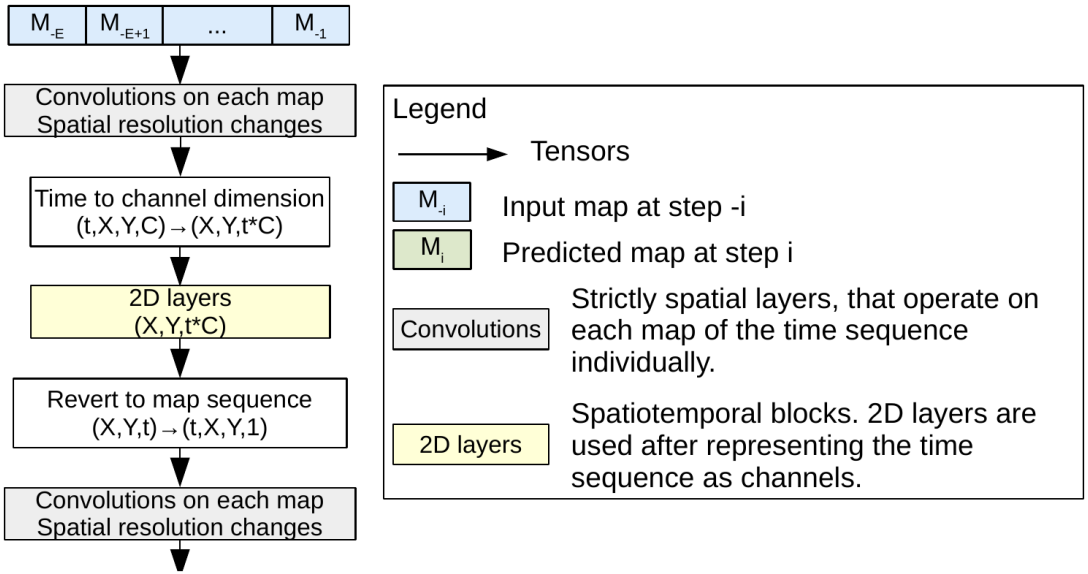
UNet3D architecture: hierarquical spatial representation widens the spatial receptive field of each predicted pixel.

Remember BiConvLSTM?

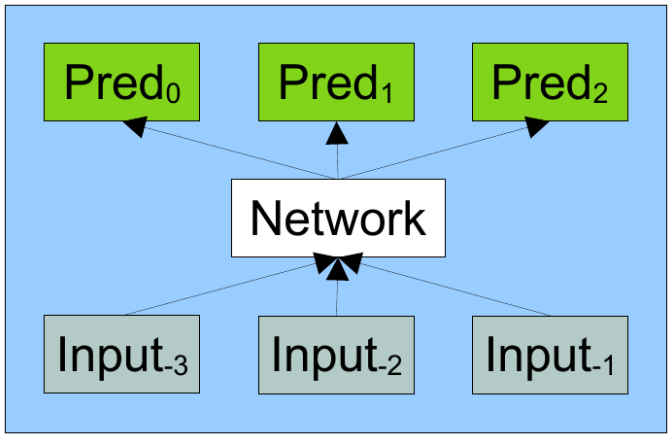


Prediction method with UNet3D

2D Convolutional networks: SimVP v2



SimVP architecture (TAN et al., 2022): The time dimension is represented in the channel dimension, thus 2D layers can be used to processes sequences of maps.

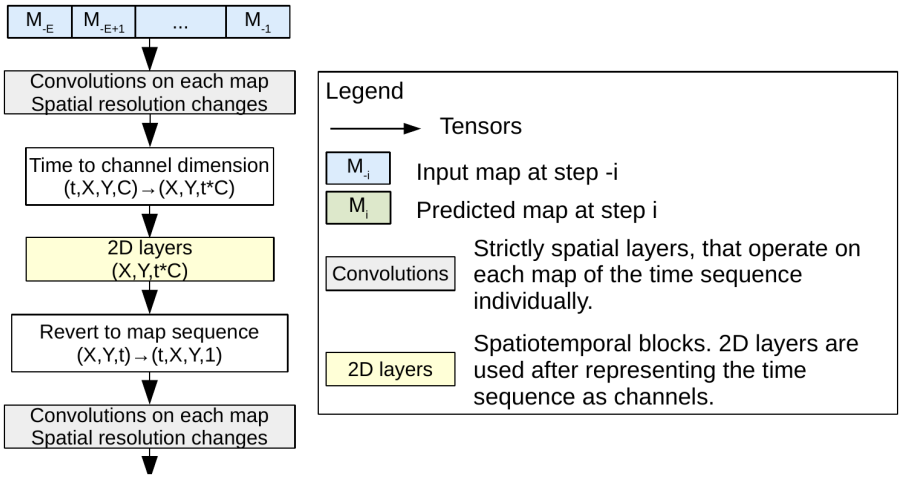


Prediction method with SimVP

The receptive field in the time dimension is the entire time sequence for each pixel.

Downside: Too many values on each neuron.

2D Convolutional networks: SimVP v2



SimVP architecture (TAN et al., 2022): The time dimension is represented in the channel dimension, thus 2D layers can be used to processes sequences of maps.

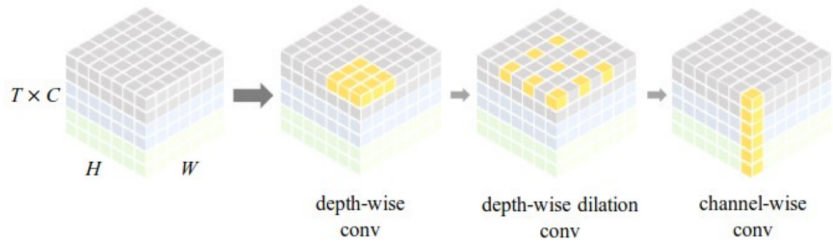
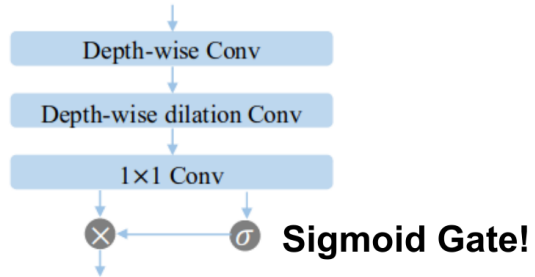


Fig. 7. The large kernel convolution in the gated spatiotemporal attention module. The yellow region denotes the receptive field.

Gated Spatiotemporal Attention Translator (gSTA)

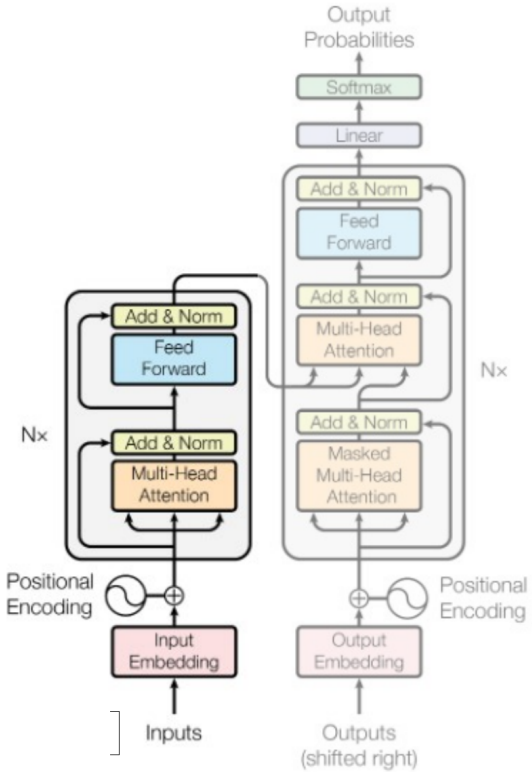
`https://github.com/chengtan9907/OpenSTL`



(Tan et al. 2022)

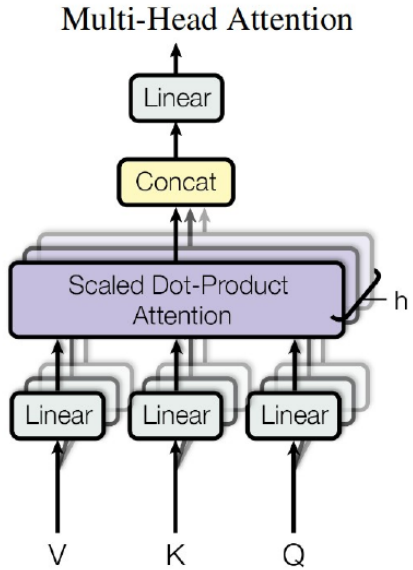
Remembering Transformer concepts

Natural Language processing
Memory: $O(\text{tokens}^2)$



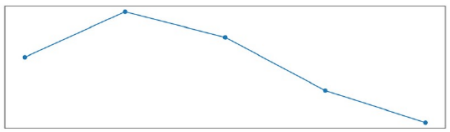
Dimensions: (tokens, channels)
Transformers architecture (Vaswani et al. 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$



Self attention:
$Q = V = K$

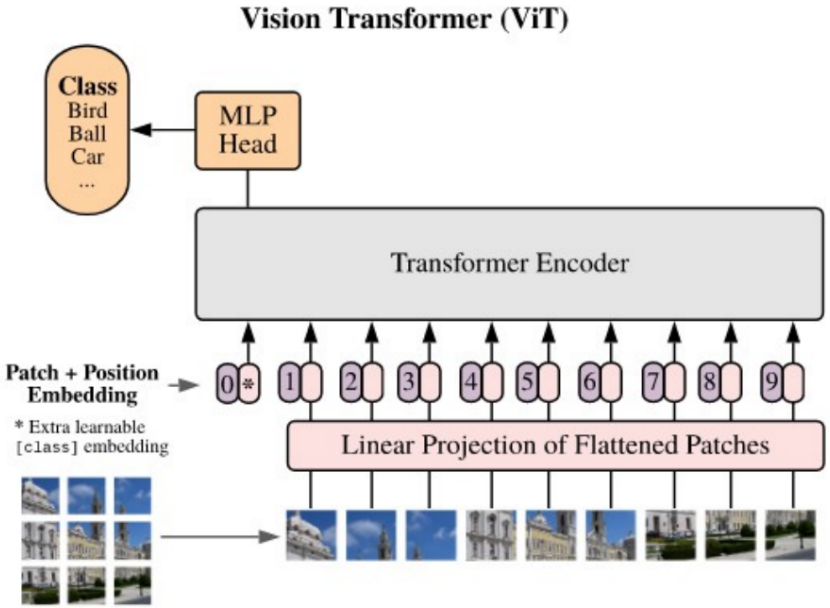
Tokens for time series ?



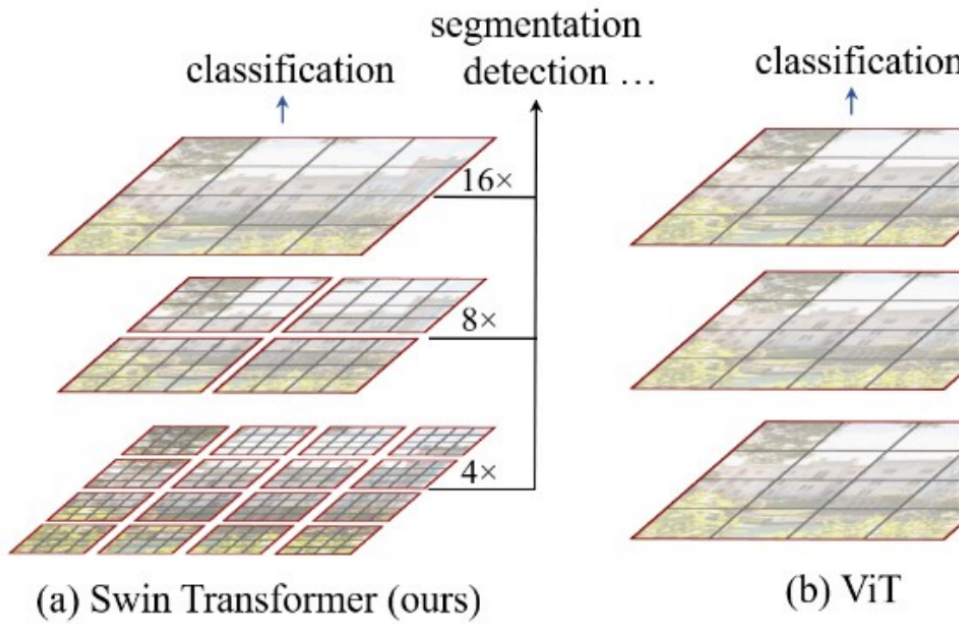
NLP tokens and an 1-D time series.

Transformers for images: ViT and Swin

From tokens describing words to tokens describing parts of an image.
These can be used with SimVP for spatiotemporal prediction!

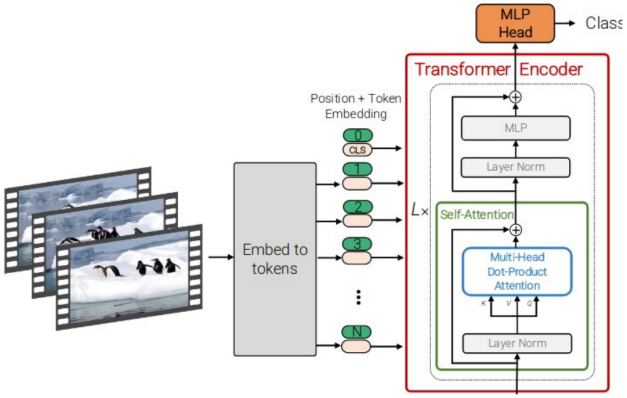


Vision Transformer (ViT): Attention is used to classify the entire image (DOSOVITSKIY et al. 2021)



Swin Transformer: Attention is used to describe the neighborhood (LIU et al. 2021)

Transformers for video: ViVit



Video Vision Transformer (ARNAB et al., 2021)

Similar to Vision Transformer (ViT), ViVit was designed to classify an entire video.

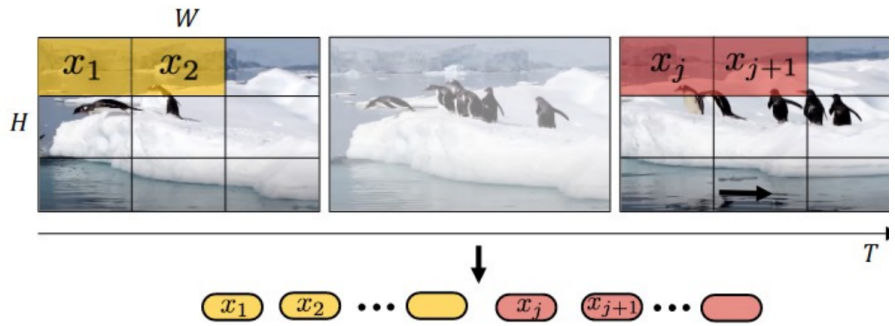


Figure 2: Uniform frame sampling: We simply sample n_t frame and embed each 2D frame independently following ViT [18].

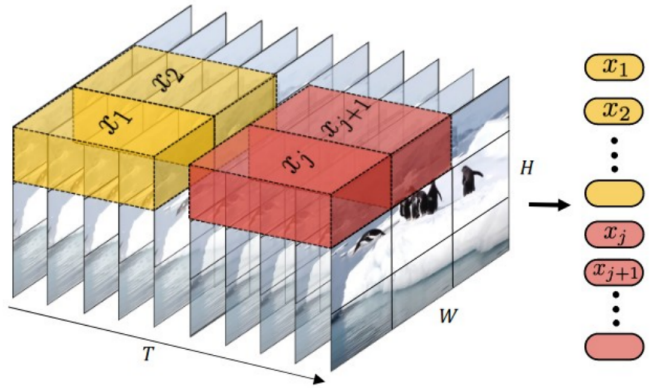


Figure 3: Tubelet embedding. We extract and linearly embed non-overlapping tubelets that span the spatio-temporal input volume. (ARNAB et al., 2021)

Transformers for spatiotemporal prediction



Model	Tokens / attention	Used for
Earthformer (Amazon, 2022)	Cuboid attention on local space-time blocks + global vectors	Precipitation nowcasting (SEVIR), ENSO
PredFormer (2026)	Pure gated transformer, factorized spatial / temporal attention; no recurrence, no convolution	Video prediction benchmarks (Moving MNIST, TaxiBJ, WeatherBench)
Pangu-Weather (Huawei, 2023)	3D Earth-specific Swin: patches over (level, lat, lon), window attention	Global medium-range forecast at 0.25°; beat ECMWF IFS on many variables
FuXi (Fudan, 2023)	U-Transformer (Swin V2), cascade of 3 models (0–5, 5–10, 10–15 days)	15-day global forecasts
Aurora (Microsoft, 2025)	Perceiver encoder (any variables / levels) + 3D Swin U-Net; 1.3 B parameters, foundation model	Fine-tuned for weather, air quality, ocean waves, cyclone tracks
AIFS (ECMWF, 2025)	Graph encoder / decoder + sliding-window transformer processor	Operational at ECMWF since Feb. 2025 (single and ensemble)

Common recipe: 2D/3D patch tokens $\rightarrow$ local window attention (Swin) to keep the cost linear in $H \times W$ $\rightarrow$ encoder–processor–decoder $\rightarrow$ autoregressive rollout (or one model per lead time), trained on ERA5.

Earthformer: `https://github.com/amazon-science/earth-forecasting-transformer`
Aurora: `https://github.com/microsoft/aurora` (`pip install microsoft-aurora`)
AIFS (Anemoi framework): `https://github.com/ecmwf/anemoi-core`

From frame regression to generative prediction

Deterministic models (previous section)

- Trained with MAE/RMSE → predict the *average* future → blur.
- Still improving: SimVP v2 and IAM4VP lead several benchmarks; PredFormer (2026) uses only gated transformers.

The generative turn

- Diffusion models: sharp forecasts and many plausible futures (ensembles).
- Flow matching / rectified flow: straight path from noise to data, plain regression loss, far fewer sampling steps.

Large video models as world simulators

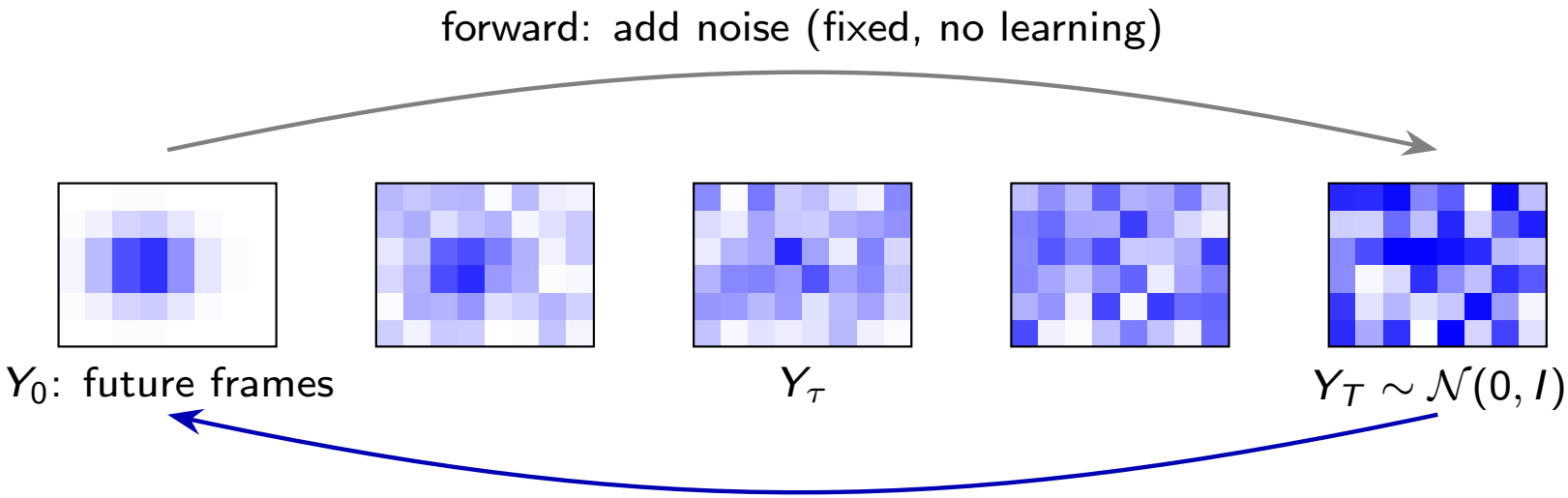
- Closed: Veo 3.1, Seedance 2.0, Genie 3.
- Open: Wan 2.2, LTX-2.5, NVIDIA Cosmos, V-JEPA 2 (predicts in embedding space).
- Evaluated as future-state predictors (WorldReasonBench, 2026).

The shift

From "predict the next frame" to **"sample possible futures"**.

(Tang et al. 2026; Lipman et al. 2023; Assran et al. 2025)

Diffusion video prediction: core idea



Training

- Sample a step τ and noise $\epsilon \sim \mathcal{N}(0, I)$.
- Noisy target: $Y_\tau = \sqrt{\bar{\alpha}_\tau}\, Y + \sqrt{1 - \bar{\alpha}_\tau}\, \epsilon$
- Predict the noise, given the past X: $\mathcal{L} = \|\epsilon - \epsilon_\theta(Y_\tau, \tau, X)\|^2$

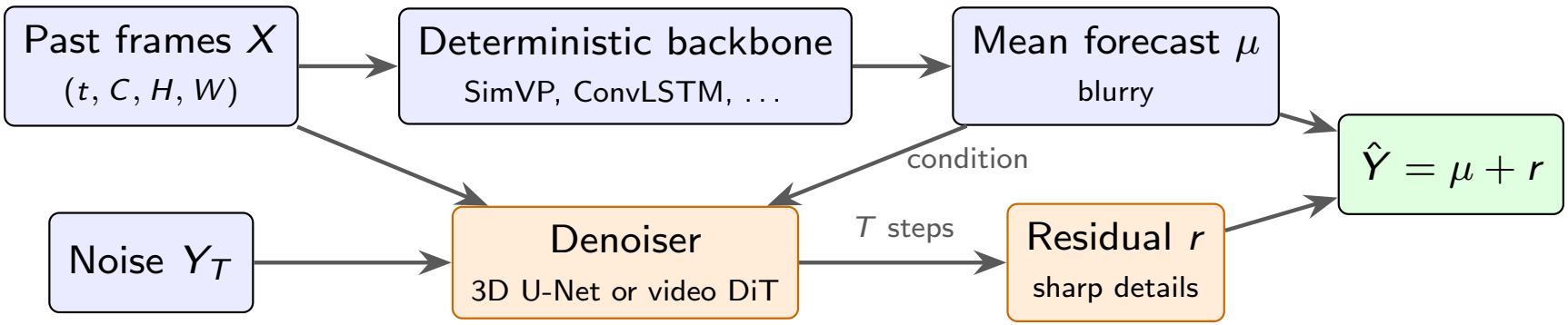
Sampling (forecast)

- Start from noise Y_T, remove a little of the predicted noise at each step.
- $T \approx 1000$ steps (DDPM) or 20–50 (DDIM).
- New noise $\rightarrow$ a new plausible future: ensembles for free.

τ is the *diffusion step*, not the forecast time t.

(Ho et al. 2020; Song et al. 2021)

Diffusion video prediction: what changes for video

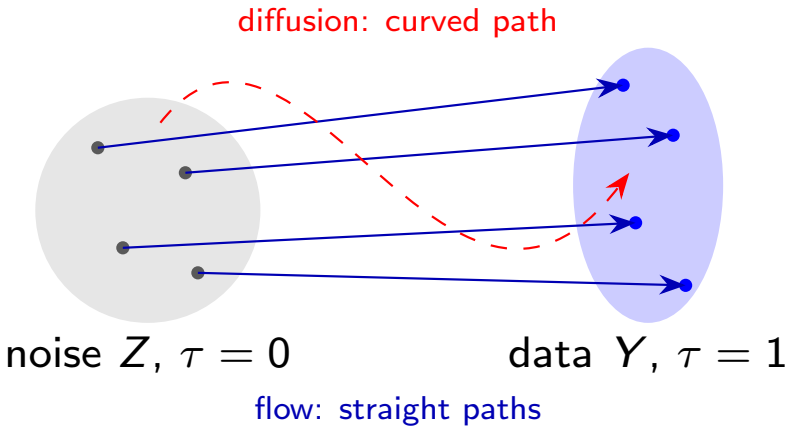


Residual diffusion: the backbone predicts the mean, diffusion only adds what the mean cannot (DiffCast, CasCast, CorrDiff).

- **The whole clip is one sample**: $Y \in \mathbb{R}^{t' \times C \times H \times W}$ is denoised jointly $\rightarrow$ temporally consistent frames.
- **Conditioning on** X: concatenate as channels or use cross-attention; τ enters through an embedding (FiLM / adaLN).
- **Denoiser** = the architectures we saw: 3D convolutions or spatiotemporal transformers.
- **Latent diffusion**: compress the clip with an autoencoder and diffuse in the latent space (cheaper).
- **Long horizons**: predict a chunk, feed it back, repeat.

(Yu et al. 2024; Rombach et al. 2022)

Flow matching video prediction: core idea



- **Path**: a straight line between a noise sample and a future clip

$$Y_\tau = (1 - \tau)\, Z + \tau\, Y, \quad \tau \in [0, 1]$$

- **Velocity** along the path is constant: $dY_\tau / d\tau = Y - Z$
- **Training**: plain regression of the velocity

$$\mathcal{L} = \| v_\theta(Y_\tau, \tau, X) - (Y - Z) \|^2$$

- **Sampling**: solve the ODE from $\tau = 0$ to 1 with Euler steps

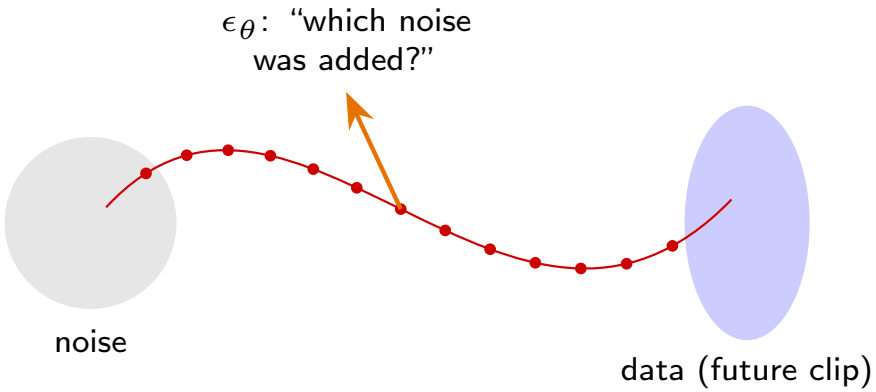
$$Y \leftarrow Y + \Delta\tau\; v_\theta(Y, \tau, X)$$

Why it matters for forecasting

Straight paths are easy to follow: **1–20 steps** instead of hundreds. Different $Z \rightarrow$ different futures (ensemble). (Lipman et al. 2023; Liu et al. 2023)

Diffusion vs. flow matching

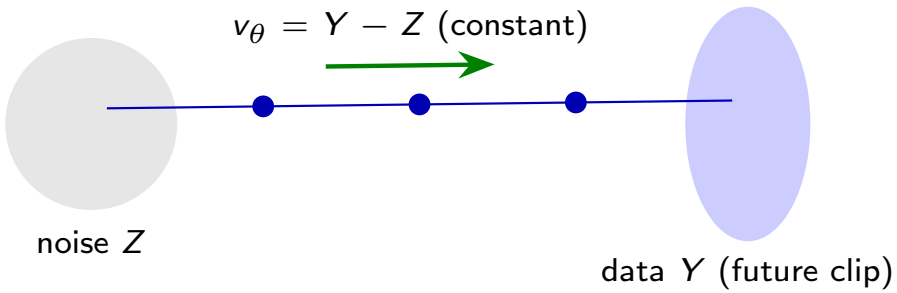
Diffusion: predict the noise



many small steps on a curved path

20–1000 steps

Flow matching: predict the velocity



few large Euler steps on a straight line

1–20 steps

	Diffusion	**Flow matching**
Network predicts	noise ϵ	velocity $Y - Z$
Path noise $\rightarrow$ data	curved (noise schedule $\bar{\alpha}_\tau$)	straight line
Sampling steps	20–1000	1–20
Extra hyperparameters	noise schedule, sampler	almost none

The same shift in weather forecasting



Nowcasting (0–6 h): radar and satellite as video

- Main recipe: **deterministic backbone + generative residual** (DiffCast, CasCast, CorrDiff).
- Flow matching for speed: FlowCast (ICLR 2026), FREUD (CVPR 2026).
- Satellite brightness temperature: diffusion on SEVIRI and GOES infrared images.

Global forecasts (up to 15 days)

- GenCast (diffusion, 50 members) beat ECMWF ENS on more than 96% of targets.
- WeatherNext 3 (Aug 2026): 64 members, hourly updates.
- WeatherNext Cyclones: track, intensity and size.

In practice

- Ensembles by default.
- Metrics beyond RMSE: CRPS, CSI, power spectra.
- Sampling time is the bottleneck.

(Yu et al. 2024; Price et al. 2025; Chase et al. 2026; Afzali Gorooh et al. 2026; Alet et al. 2026)

Outline

1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

Earth foundation embeddings: what is available

Product	Type	Dim	Res.	Years / access
AlphaEarth (Google)	pixel	64	10 m	2017–, annual; Earth Engine, Source Coop.
TESSERA (Cambridge)	pixel	128	10 m	2017–2025*, annual
Presto	pixel	128	10 m	2019–2020, Togo only
Clay	patch	768	5.1 km	2018–2023*; Source Coop.
Major TOM	patch	2048	∼3 km	2015–2024*; Hugging Face
Earth Index	patch	384	320 m	2024; Source Coop.
Copernicus-Embed	patch	768	0.25°	2021; Hugging Face
DINOv3-Sat (Meta)	patch	1024	∼10 m	weights only, no precomputed embeddings; Hugging Face

* sparse coverage. TorchGeo has data loaders for all of them.

Foundation models (run them yourself)

- Surface: Prithvi-EO-2.0, TerraMind, OlmoEarth, Galileo, DOFA, Clay, DINOv3-Sat.
- Atmosphere: Aurora, Prithvi WxC.

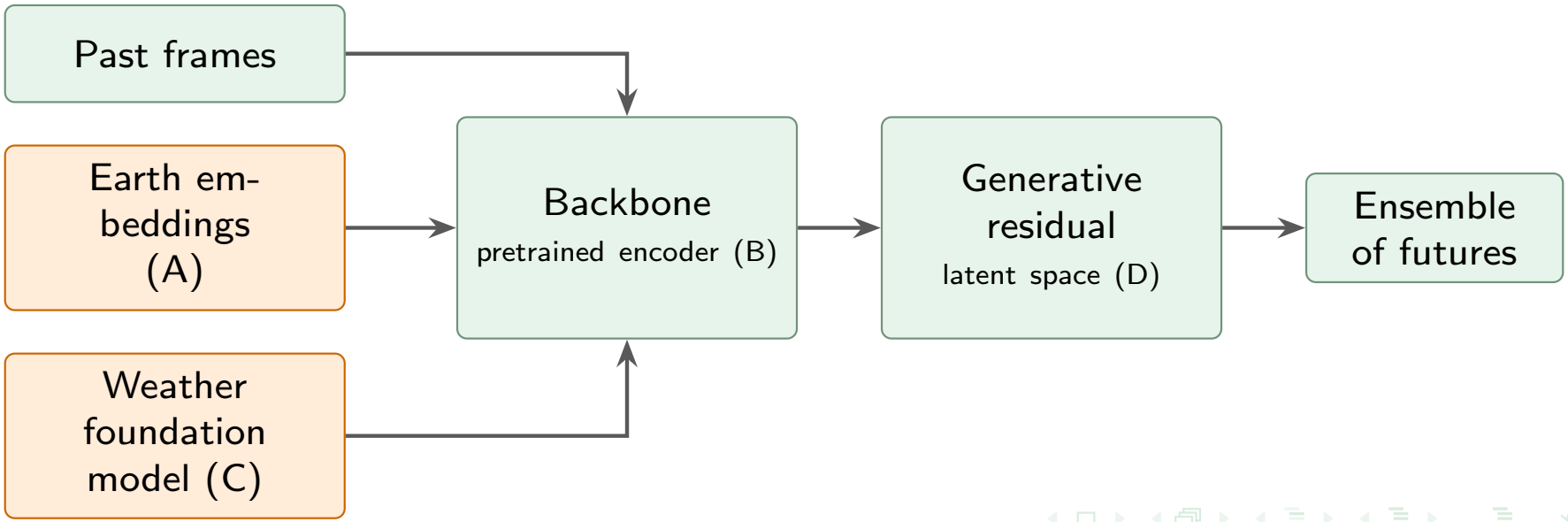
Watch out for

- Annual snapshots vs. minutes-scale weather.
- Mixed licenses (AlphaEarth weights are closed).
- Different data types and grids.

(Earth Embeddings as Products, 2026)

How embeddings change the landscape

- Embeddings become a **data layer**: download a vector per pixel instead of training an encoder.
- Good results with **few labels** (k-NN or a linear layer on top).
- New models arrived roughly every two weeks in early 2026.
- Surface knowledge (terrain, land cover, water, cities) becomes a ready-made input for weather models.



Integrating embeddings in a forecast model



- **A. Static conditioning**
  - Pool the embeddings to the forecast grid (e.g. 10 m $\rightarrow$ 2 km), reduce with PCA, add as extra input channels (or FiLM).
  - Cheapest option: works with any model from the previous section.
- **B. Pretrained encoder**
  - Replace the encoder with an Earth observation foundation model (TerraTorch, timm).
- **C. Dynamic context**
  - Forecasts or latents from Aurora / Prithvi WxC as conditioning, so the model knows the large-scale weather.
- **D. Latent generation**
  - Run diffusion / flow matching in a pretrained latent space to reduce cost.

Summary before the hands on



1. The field moved from **deterministic frame regression** to **generative sampling of possible futures**.
2. Weather systems converged on one recipe: **deterministic backbone + generative residual**, evaluated with CRPS, CSI and spectra.
3. **Earth embeddings** turn the land surface into a ready-made input layer.
4. Main gaps: time-scale mismatch (annual vs. minutes), licensing, evaluation cost.

Next: hands on

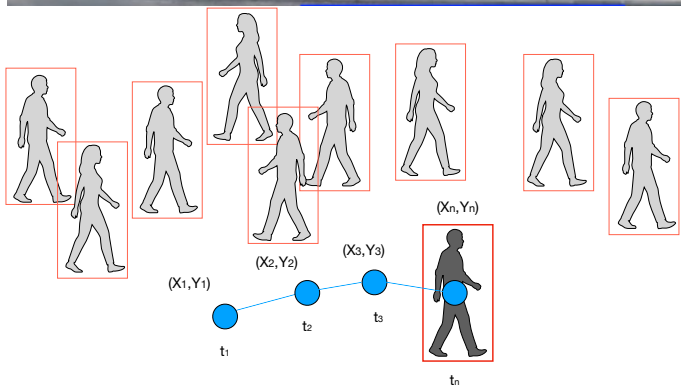
The OpenSTL models are the **deterministic backbone**. A generative residual or embedding channels can be added on top of them.

Outline

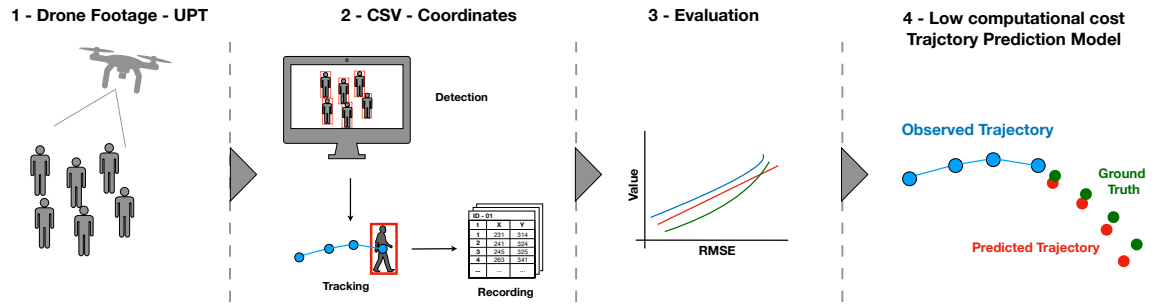
1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

Human Trajectory Prediction on UAV Images: A Comparative Study

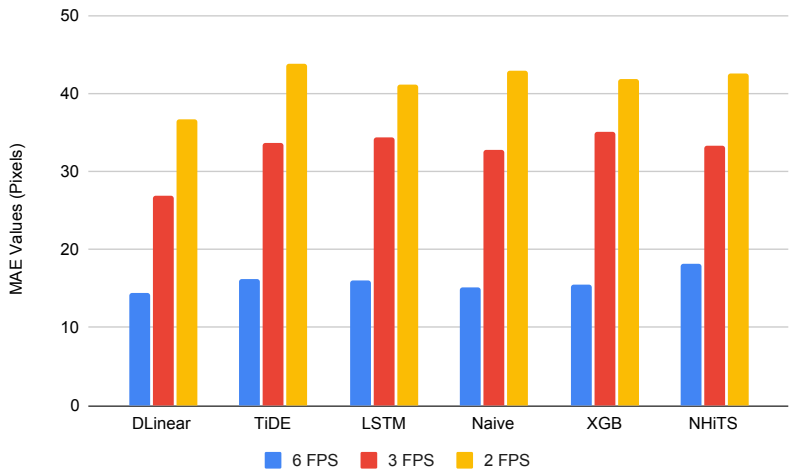




Each tracked person becomes a sequence of bounding box centers (x, y): coordinates in time (approach 1).



Pipeline: drone footage → detection and tracking → coordinates → trajectory prediction.



MAE (pixels) by model and frame rate: the simple linear model (DLinear) is the best.

da Hora RDM, Santos DR, de Paulo MCM, Ferrari F, Feitosa RQ, Rosa PFF (2026) Human Trajectory Prediction on UAV Images: A Comparative Study. Int Arch Photogramm Remote Sens Spatial Inf Sci XLIX-B2-2026:755–761. https://doi.org/10.5194/isprs-archives-XLIX-B2-2026-755-2026

Outline

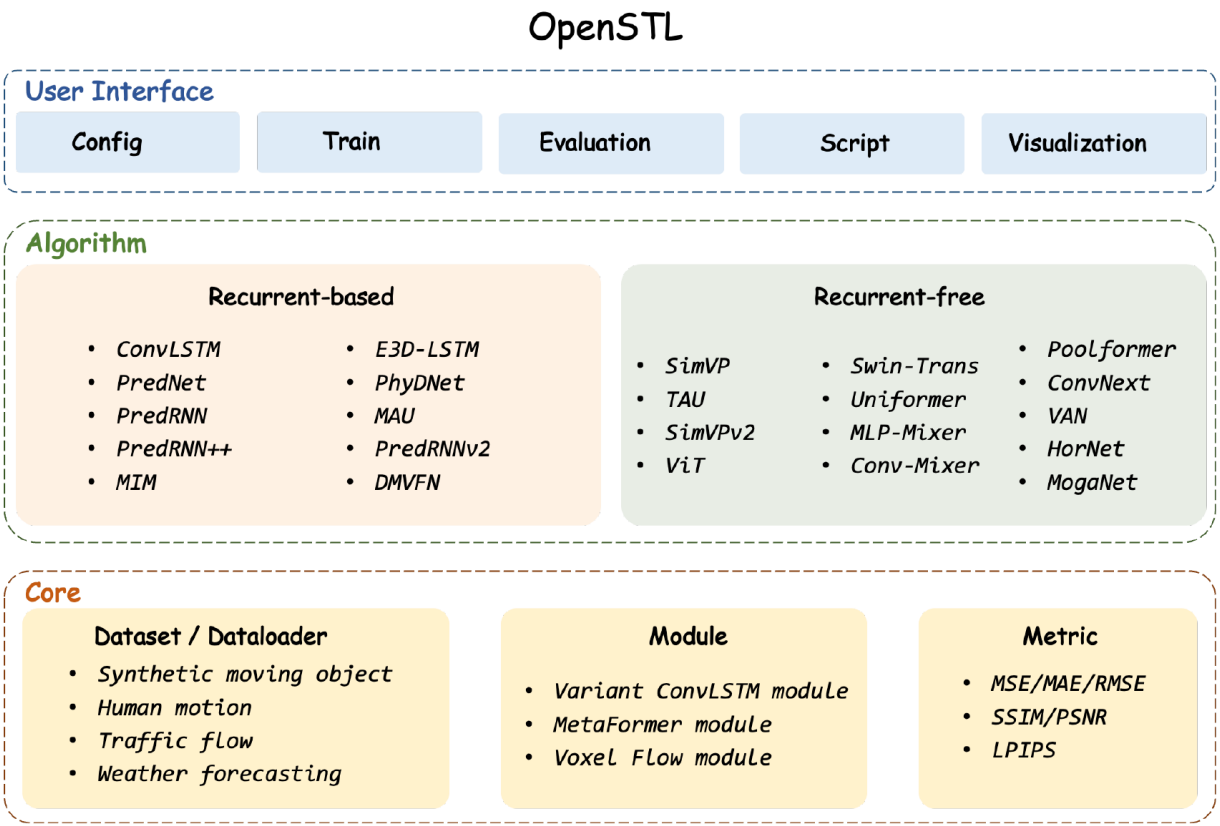
1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

Spatiotemporal forecasting libraries

Library	Description	Points / trajectories	Sensor graphs	Maps (grids)
darts	Time series forecasting with one API for all models (ARIMA, XGBoost, N-HiTS, TiDE, TFT, ...).	✓	–	–
neuralforecast	Nixtla's deep learning forecasters (N-HiTS, TFT, PatchTST, ...), scales to many series.	✓	–	–
tsl	Torch Spatiotemporal: spatiotemporal graph neural networks on PyG + Lightning, handles missing data.	∼	✓	∼
PyG Temporal	Recurrent and attention graph layers for signals on graphs (older, less active).	∼	✓	∼
OpenSTL	Video-style predictive learning: ConvLSTM, PredRNN, SimVP, ... with Lightning. **Our hands on.**	–	–	✓
PredBench	Benchmark: 12 methods on 15 datasets, long-term and robustness tests.	–	–	✓
PhysicsNeMo	NVIDIA training framework: FourCastNet, SFNO (sphere), U-Net, CorrDiff, MeshGraphNet.	–	∼	✓
Anemoi	ECMWF framework for operational weather models (datasets, graphs, training, inference).	–	–	∼
Earth2Studio	Runs pretrained weather models (inference only) with ERA5, GFS, IFS data sources.	–	–	∼

✓ native ∼ workaround or limited (pixels as graph nodes, weather data only, inference only) – not supported

OpenSTL (Pytorch+lightning)



Possibilities

- Create your dataloader and train a model.
- Use the models in your project.
- Create new models.

What's the difference?

- Learning capacity
- Memory
- Forecast procedure
- Training speed
- Prediction speed

(Tan et al. 2023)

OpenSTL benchmark datasets (results)

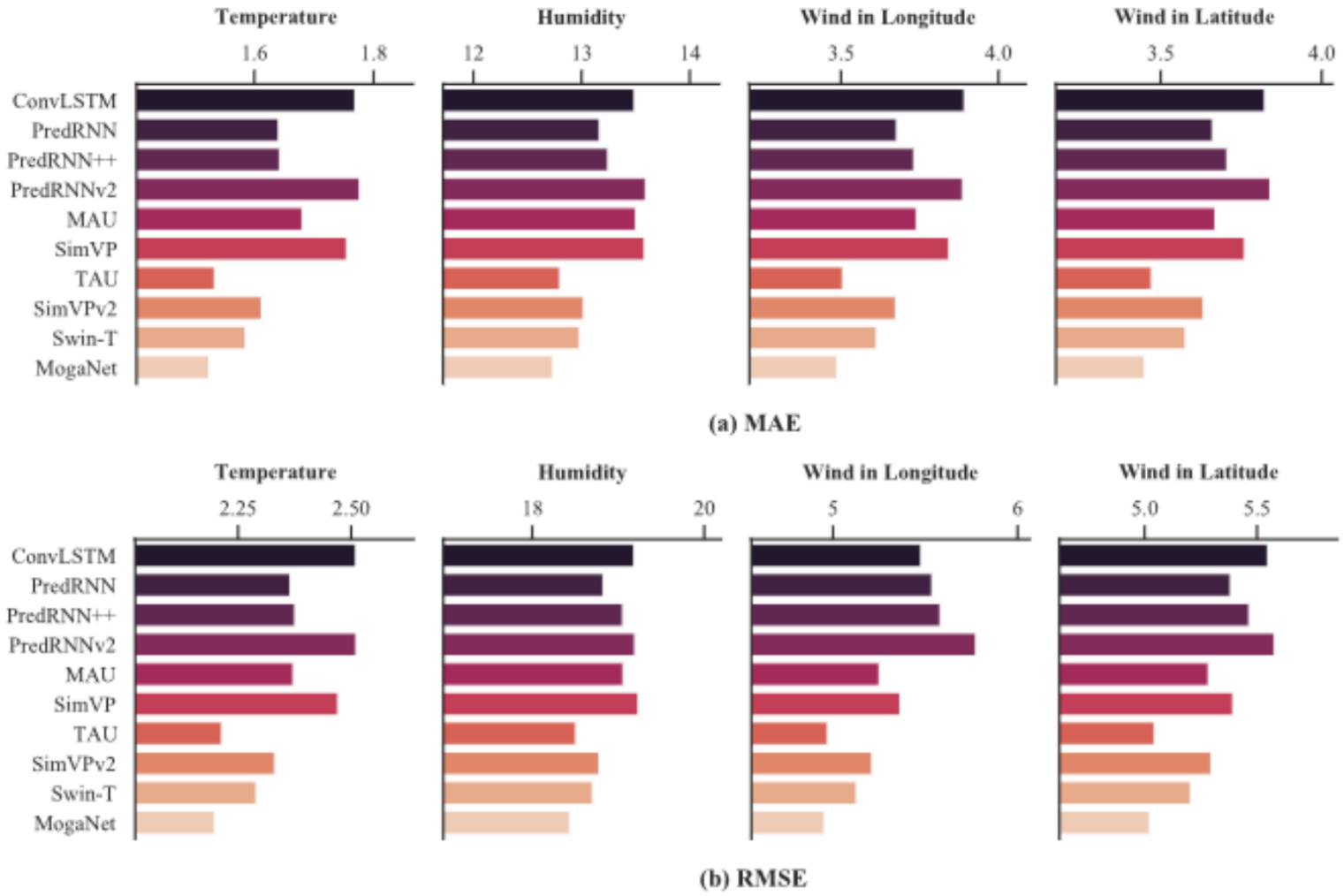


Figure 17: The (a) MAE and (b) RMSE metrics of the representative approaches on the four weather forecasting tasks in WeatherBench (Muti-variable setting).

(Tan et al., 2023)

Google Colab code



- **OpenSTL in Global Ionosphere Maps prediction (hands on)**
  - https://github.com/mauriciodev/tec_forecast/blob/main/examples/OpenSTL_GIM.ipynb
- OpenSTL custom dataloader tutorial
  - https://github.com/chengtan9907/OpenSTL
  - https://colab.research.google.com/drive/19uShc-1uCcySrjrRP3peXf2RUNVzCjHh?usp=sharing
- GIM Forecast with PyTorch (Paulo et al, 2023)
  - https://github.com/mauriciodev/tec_forecast/blob/main/extra/tec_forecast_torch.ipynb
- GIM Forecast with Tensorflow/Keras (Paulo et al, 2023)
  - https://github.com/mauriciodev/tec_forecast/blob/main/examples/tec_forecast_tf2_lab.ipynb

Outline

1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

Loss functions and evaluation metrics



Numerical prediction (Regression)

$$MAE = \frac{\sum_{i=1}^{n} |y_i - \hat{y}_i|}{n}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}}$$

$$Huber = \frac{\sum_{i=1}^{n} L_{huber}(y, \hat{y})}{n}$$

where

$$L_{huber}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{for } |y - \hat{y}| < \delta \\ \delta\left(|y - \hat{y}| - \frac{1}{2}\delta\right) & \text{otherwise} \end{cases}$$

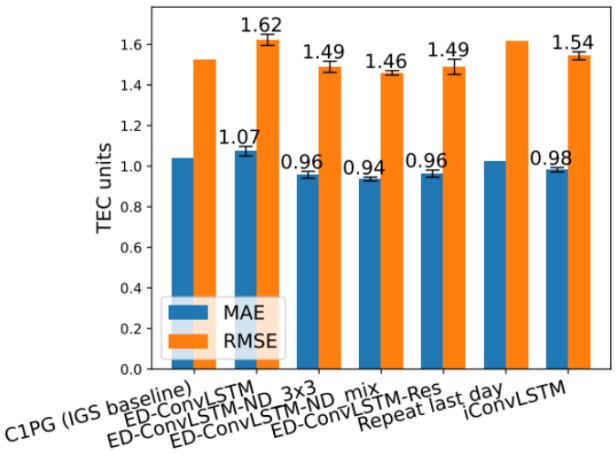
$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

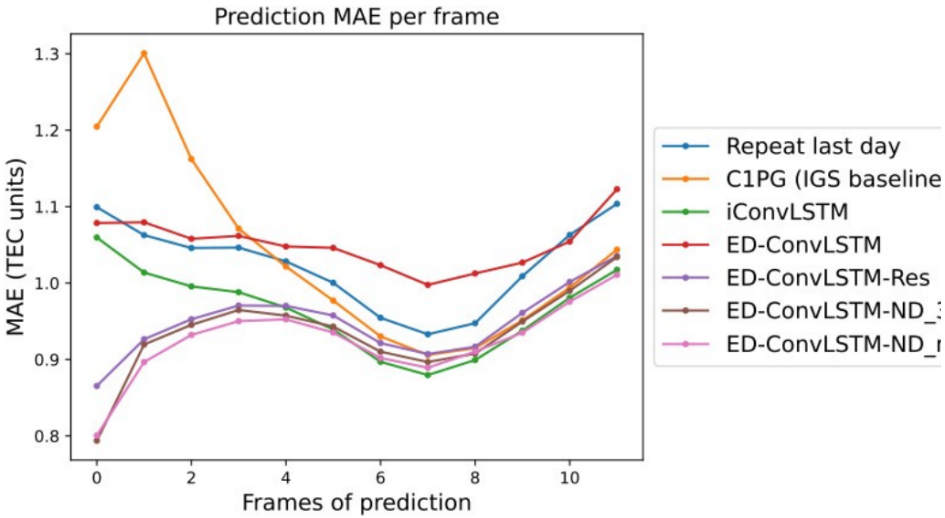
Categorical prediction

- Categorical cross entropy loss
- Focal loss
- Precision, recall, F-score
- Intersection over union (IOU)

n: Number of samples in a batch
y: Values from the training sample;
$\hat{y}$: Values predicted with the network.

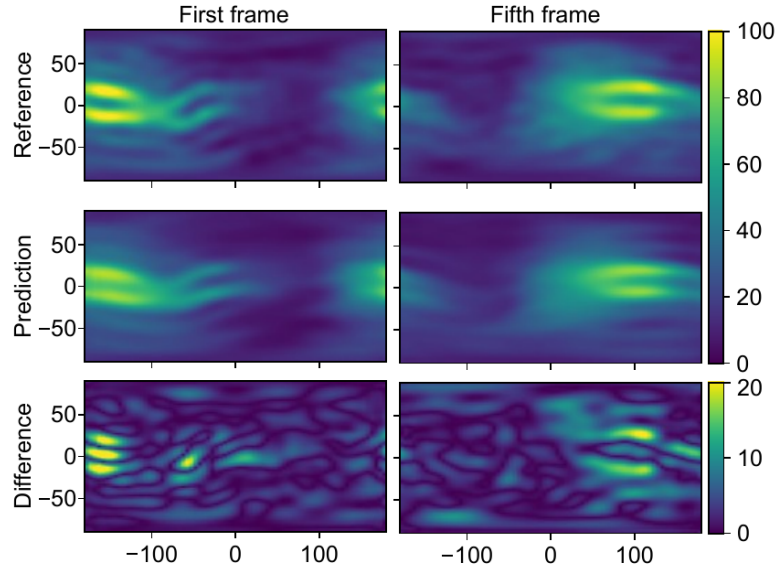
Exploring the results

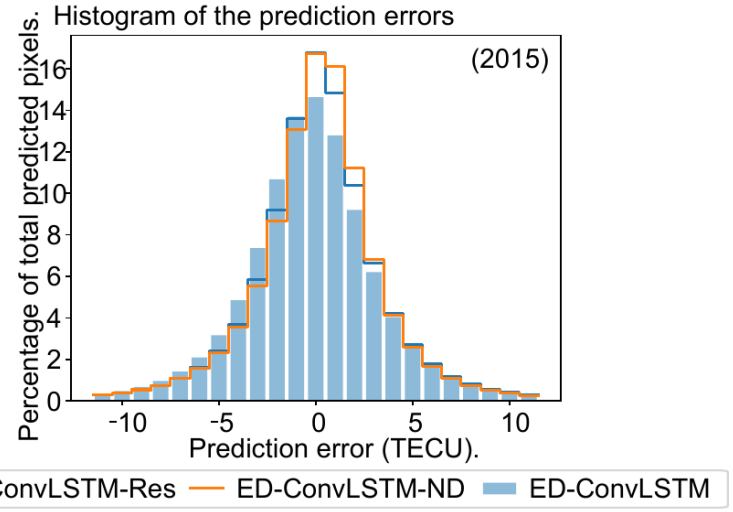




Frame-wise MAE.

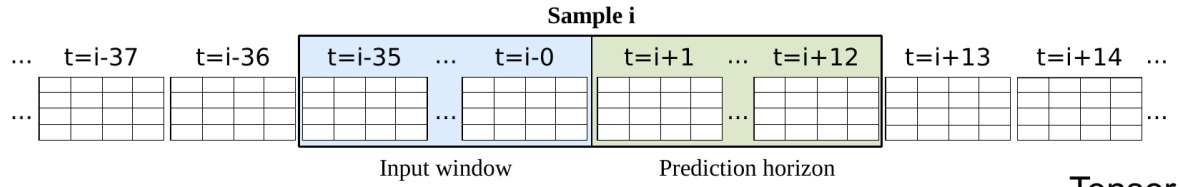
Difference between prediction and reference





Data set construction: What do we measure?

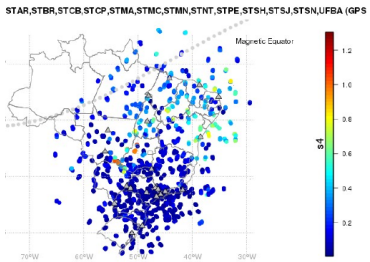
This is what we want:



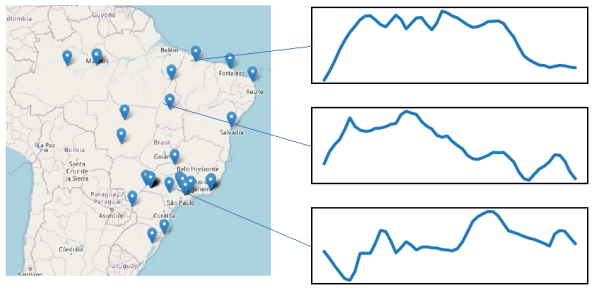
Tensor dimensions: (t,c,x,y)

Pandas: Temporal resampling
GDAL or PyKrige: Spatial interpolation

But sometimes this is what we have:



Scattered points with a timestamp.



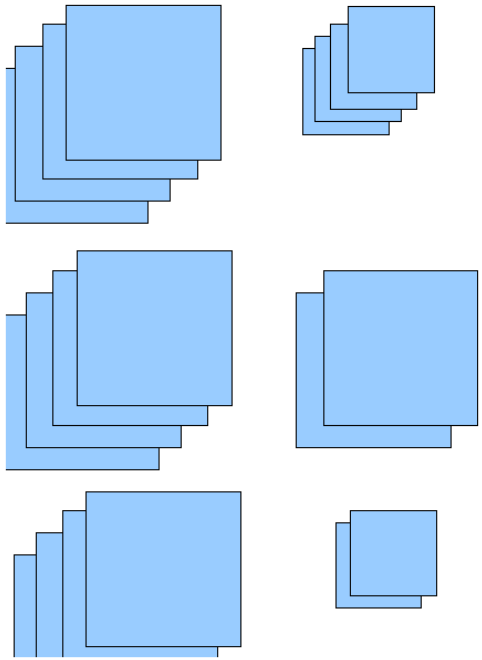
Time series on multiple stations.

Among others (1D? Graphs?)

Convolutional layers dimensions

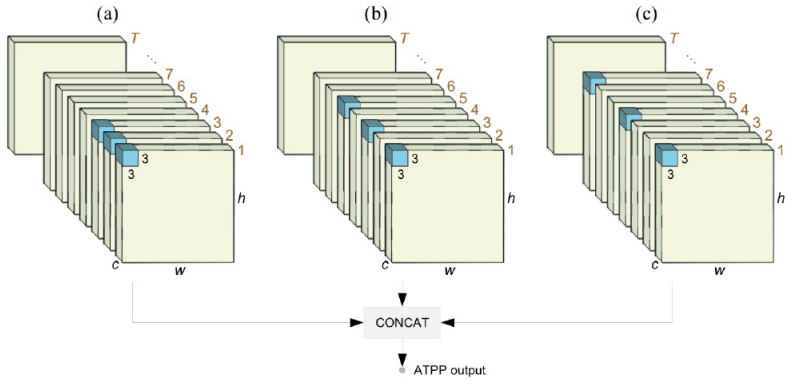


- Popular pooling options:
  - Max
  - Average
- Spatial pooling
  - $(t, x, y, c) \rightarrow (t, x/2, y/2, c)$
- Temporal pooling
  - $(t, x, y, c) \rightarrow (t/2, x, y, c)$
- Spatiotemporal pooling (uncommon)
  - $(t, x, y, c) \rightarrow (t/2, x/2, y/2, c)$
- **Convolution with stride can change resolution too.**
- Spatial and temporal resolution changes must make sense for the event.

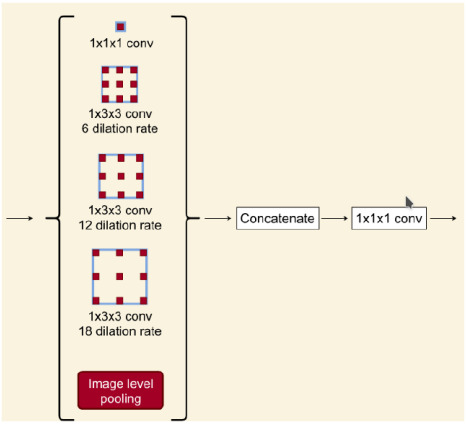


Convolutional layers dimensions

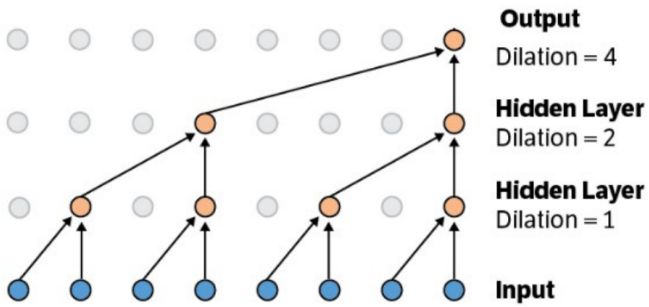
- **Dilated convolution**
  - Spatial
  - Temporal
  - Spatiotemporal (uncommon)
- **Deformable**



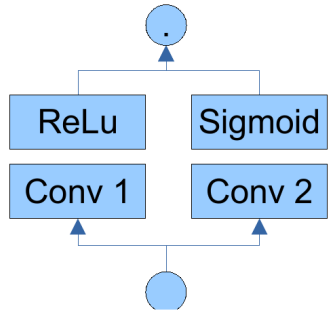
UNet 3D Atrous temporal pyramid pooling (Rogozinski, 2021)



Broad-UNet Atrous spatial pyramid pooling (Fernández, 2021)



Wavenet's sequence of time dilated convolutions (Borovykh, 2018)



Sigmoid gates control the output of a block.

Outline

1. Introduction
2. Spatiotemporal forecast basics
3. Data set construction
4. Architectures
5. Foundation models (2026)
6. Examples
7. Hands on
8. Other topics
9. References

References I

Afzali Gorooh V, Delle Monache L, Axisa D, Sengupta A, Zhang Z, Ralph FM (2026) Deterministic nowcasting of geostationary satellite infrared brightness temperature using 3D U-Net diffusion model. Scientific Reports 16:4191. https://doi.org/10.1038/s41598-025-34207-9

Alahi A, Goel K, Ramanathan V, Robicquet A, Fei-Fei L, Savarese S (2016) Social LSTM: Human Trajectory Prediction in Crowded Spaces. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp 961–971

Alet F, Andersson TR, Titley H, et al. (2026) Operational tropical cyclone forecasting with AI. Nature. https://doi.org/10.1038/s41586-026-10953-2

Arnab A, Dehghani M, Heigold G, Sun C, Lucic M, Schmid C (2021) ViViT: A Video Vision Transformer. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV). IEEE, Montreal, QC, Canada, pp 6816–6826

Assran M, et al. (2025) V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning

Auer A, Podest P, Klotz D, Böck S, Klambauer G, Hochreiter S (2025) TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning. NeurIPS

Beck M, Pöppel K, Spanring M, Auer A, Prudnikova O, Kopp M, Klambauer G, Brandstetter J, Hochreiter S (2024) xLSTM: Extended Long Short-Term Memory

Bi K, Xie L, Zhang H, Chen X, Gu X, Tian Q (2023) Accurate medium-range global weather forecasting with 3D neural networks. Nature 619:533–538

Bodnar C, Bruinsma WP, Lucic A, et al. (2025) A foundation model for the Earth system. Nature 641:1180–1187

Borovykh A, Bohte S, Oosterlee CW (2018) Conditional Time Series Forecasting with Convolutional Neural Networks. arXiv:170304691 [stat]

Chase RJ, Haynes K, Ver Hoef L, Ebert-Uphoff I (2026) How to use score-based diffusion in earth system science: A satellite nowcasting example. Artificial Intelligence for the Earth Systems. arXiv:2505.10432

Chen L, Zhong X, Zhang F, Cheng Y, Xu Y, Qi Y, Li H (2023) FuXi: a cascade machine learning forecasting system for 15-day global weather forecast. npj Climate and Atmospheric Science 6:190

References II

da Hora RDM, Santos DR, de Paulo MCM, Ferrari F, Feitosa RQ, Rosa PFF (2026) Human Trajectory Prediction on UAV Images: A Comparative Study. Int Arch Photogramm Remote Sens Spatial Inf Sci XLIX-B2-2026:755–761. https://doi.org/10.5194/isprs-archives-XLIX-B2-2026-755-2026

Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, Uszkoreit J, Houlsby N (2021) An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Earth Embeddings as Products: Taxonomy, Ecosystem, and Standardized Access (2026). arXiv:2601.13134

Feng L, Tung F, Ahmed MO, Bengio Y, Hajimirsadeghi H (2024) Were RNNs All We Needed? arXiv:2410.01201

Fernández JG, Mehrkanoon S (2021) Broad-UNet: Multi-scale feature learning for nowcasting tasks. Neural Networks 144:419–427. https://doi.org/10.1016/j.neunet.2021.08.036

Gao Z, Shi X, Wang H, Zhu Y, Wang B, Li M, Yeung D-Y (2022) Earthformer: Exploring Space-Time Transformers for Earth System Forecasting

Herzen J, Lässig F, Piazzetta SG, et al. (2022) Darts: User-Friendly Modern Machine Learning for Time Series. Journal of Machine Learning Research 23(124):1–6

Ho J, Jain A, Abbeel P (2020) Denoising Diffusion Probabilistic Models. NeurIPS

Lang S, Alexe M, Chantry M, et al. (2024) AIFS – ECMWF's data-driven forecasting system. arXiv:2406.01465

Liu X, Gong C, Liu Q (2023) Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. ICLR

Li Y, Yu R, Shahabi C, Liu Y (2018) Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. ICLR

Lipman Y, Chen RTQ, Ben-Hamu H, Nickel M, Le M (2023) Flow Matching for Generative Modeling. ICLR

Li K, Li X, Wang Y, He Y, Wang Y, Wang L, Qiao Y (2024) VideoMamba: State Space Model for Efficient Video Understanding

Li S, Wang Z, Liu Z, Tan C, Lin H, Wu D, Chen Z, Zheng J, Li SZ (2023) Efficient Multi-order Gated Aggregation Network

References III

Nie Y, Nguyen NH, Sinthong P, Kalagnanam J (2023) A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR

Orvieto A, Smith SL, Gu A, Fernando A, Gulcehre C, Pascanu R, De S (2023) Resurrecting Recurrent Neural Networks for Long Sequences. ICML

Paulo MCM de, Marques HA, Feitosa RQ, Ferreira MP (2023) New encoder–decoder convolutional LSTM neural network architectures for next-day global ionosphere maps forecast. GPS Solut 27(2):95. https://doi.org/10.1007/s10291-023-01442-4

Paulo MCM de, Marques HA, Ferreira MP, Feitosa RQ (2022) Redes neurais ConvLSTM CD aplicadas à previsão de mapas globais de ionosfera. In: Informações geoespaciais para o monitoramento do sistema terra visando o desenvolvimento sustentável. Curitiba, Brasil

Peng B, Alcaide E, Anthony Q, et al. (2023) RWKV: Reinventing RNNs for the Transformer Era. Findings of EMNLP

Price I, Sanchez-Gonzalez A, Alet F, et al. (2025) Probabilistic weather forecasting with machine learning. Nature 637:84–90

Rogozinski M, Martinez JAC, Feitosa RQ (2021) 3D convolution for multidate crop recognition from multitemporal image sequences. International Journal of Remote Sensing 0(0):1–22. https://doi.org/10.1080/01431161.2021.1976876

Rombach R, Blattmann A, Lorenz D, Esser P, Ommer B (2022) High-Resolution Image Synthesis with Latent Diffusion Models. CVPR

Schmude J, Roy S, Trojak W, et al. (2024) Prithvi WxC: Foundation Model for Weather and Climate. arXiv:2409.13598

Song J, Meng C, Ermon S (2021) Denoising Diffusion Implicit Models. ICLR

Tan C, Gao Z, Li SZ, Li S (2022) SimVP: Towards Simple yet Powerful Spatiotemporal Predictive Learning

Tan C, Li S, Gao Z, Guan W, Wang Z, Liu Z, Wu L, Li SZ (2023) OpenSTL: A Comprehensive Benchmark of Spatio-Temporal Predictive Learning

References IV

Tang S, Li C, Zhang P, Tang R (2023) SwinLSTM: Improving Spatiotemporal Prediction Accuracy using Swin Transformer and LSTM

Tang Y, Dong P, Tang Z, Chu X, Liang J (2024) VMRNN: Integrating Vision Mamba and LSTM for Efficient and Accurate Spatiotemporal Forecasting. CVPR Workshops, pp 5663–5673

Tang Y, Qi L, Li X, Ma C, Yang M-H (2026) Video Prediction Transformers Without Recurrence or Convolution. Transactions on Machine Learning Research

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention Is All You Need

Wu Z, Pan S, Long G, Jiang J, Zhang C (2019) Graph WaveNet for Deep Spatial-Temporal Graph Modeling. In: Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI), pp 1907–1913

Yu D, Li X, Ye Y, Zhang B, Luo C, Dai K, Wang R, Chen X (2024) DiffCast: A Unified Framework via Residual Diffusion for Precipitation Nowcasting. CVPR